FORGE - an AI integrated E Commerce System

Project documentation for Object-Oriented Programming

Group members

- Mark Reeve P. Consul
- Sarigumba Generry
- Blessy May Manila Aldaya
- Melca Alegado
- Aron Jef Tugahan

Course and section: BSIT - 2 Section 06

Instructor: Ms. Mary Clarence Bababtid

Documentation and verification date: 24 October 2026

This report explains the implemented FORGE desktop store, its object-oriented design, class relationships, interfaces and tested workflows. It includes current UML sheets and screenshots. Payments and fulfillment are course demonstrations; online AI features depend on the configured external provider.

1. Project Title

FORGE - an AI integrated E Commerce System

FORGE is a single-store desktop application for customers and administrators. The project applies Java object-oriented programming to shopping, inventory, orders and assisted catalog management.

Contents

1. Project Title - page 1

2. Project Description - page 3

3. Problem Statement - page 3

4. Objectives - page 3

5. System Features - page 4

6. OOP Implementation - page 6

7. UML Class Diagram - page 8

8. GUI Screenshots - page 24

9. Testing - page 37

10. Conclusion - page 39

UML sheets are landscape A3 so class names and operations remain readable. Interface screenshots are landscape A4. SVG diagrams and editable Markdown accompany the PDFs. The evidence is a dated snapshot. Later code changes require regenerated diagrams and relevant tests.

2. Project Description

FORGE uses Java 21 source compatibility, JavaFX 21.0.6, Maven and SQLite through JDBC. Gson handles AI request and response JSON. JUnit 5 supplies automated testing. The application starts in a resizable window and opens a storefront that guests can browse before signing in.

Customers browse technology categories, search and filter products, inspect photos and specifications, save favorites, manage a cart, place simulated orders and inspect their own purchase records. Administrators maintain inventory, process orders and view customer summaries and recorded sales. FORGE AI adds catalog assistance, image matching, review summaries and reviewed administrative content drafts.

The architecture separates JavaFX views, application services, domain models and JDBC repositories. ShopApp explicitly constructs dependencies; Services carries those service instances. Models contain no JavaFX or JDBC imports. UI handlers collect input and invoke services, while permissions and transaction rules remain in the services.

SQLite stores accounts, products, photos, favorites, profiles, addresses, saved carts, purchase snapshots, reviews, fulfillment events and AI change audits. Monetary values use BigDecimal in Java and integer cents in storage. Dates are stored as instants and grouped or displayed in the selected local time zone where applicable.

The application is designed for a trusted local desktop and a small catalog. It has no remote server, account registration, real payment gateway, live delivery integration or cross-device synchronization. Anyone who can modify the local application or database can alter the demonstration data; the desktop database is not a production security boundary.

3. Problem Statement

A small technology store needs consistent product information, current availability and reliable order records. Customers need to locate relevant equipment and understand what is actually listed, while staff need to maintain stock and product details without losing purchase history.

Uncoordinated updates can leave a cart at an outdated price, deduct stock without saving an order or overwrite a product while another edit is pending. A text-only assistant can also describe products without presenting the corresponding records, or invent specifications that have never been verified.

FORGE addresses these problems through validated domain objects, service-level role checks, transactional checkout and typed catalog retrieval. It presents current records as product cards, keeps historical purchase snapshots and requires review before AI-proposed product content or status changes are saved.

4. Objectives

- Apply encapsulation, inheritance, polymorphism and abstraction through connected Java workflows.
- Provide a readable GUI for browsing, purchasing and administering a technology catalog.
- Validate required fields, quantities, prices, permissions and order transitions outside the GUI as well as inside it.
- Preserve stock, cart and receipt consistency through JDBC transactions and immutable snapshots.
- Use AI for assistance while retaining local control over records, permissions and approved writes.
- Verify major success and failure cases, and document the actual classes and their relationships.

5. System Features

Customer shopping

Catalog and product details. Guests and customers browse PC Sets, PC Parts, Laptops, Phones and Tablets, Peripherals and Accessories, and Other Electronics. Text search, category and price filters, supported specification filters and sorting narrow current records. Detail views show a main image, up to four additional photos, description, optional specifications and configurations identified by stored catalog facts.

Favorites and profiles. Customers can save products, edit a display name and maintain delivery addresses. Guest favorites are session data. Customer favorites and addresses persist in SQLite and are scoped to the account. Saving a favorite preserves the browsing position.

Cart and checkout. Guests can assemble a session cart. Customer carts persist and merge guest selections on sign-in. Restoration rechecks availability, quantities and prices and reports adjustments. Customers enter delivery details and choose cash on delivery or a simulated wallet result. Checkout rejects changed prices or unavailable stock and commits the order, stock deduction and saved-cart clearing together. Signing in opens the shop; a restored cart does not force navigation to the cart page.

Orders and reviews. Customers view their own recorded orders, use a reviewed Buy again flow and see locally recorded fulfillment events. Eligible customers can cancel their own confirmed unpaid orders. Verified-purchase reviews use a rating from one to five and validated text. These workflows do not contact a real delivery or payment provider.

Administration

Inventory maintenance. Administrators add and edit products, upload photos, enter specifications, adjust stock or price, activate or deactivate records and perform supported bulk actions. Deletion uses a retained deleted marker: the product disappears from ordinary listings, but historical receipts and audit references remain valid. Deactivation is appropriate for an item that may return to sale.

Order processing. Administrators move an order through Confirmed, Packed, Shipped and Delivered. Shipping requires a recorded carrier and tracking number. Valid cancellation restores stock once and records a simulated refund when applicable. Payment collection is recorded separately from delivery. Invalid or stale transitions are rejected.

Overview and reports. The dashboard reads local inventory, orders and customer summaries. The revenue graph supports periods, paid revenue versus all order value, point inspection, comparisons and animation. Integer cents are aggregated before chart coordinates are converted to floating point. Paid revenue uses PAID records, grouped by order creation date; it is not a bank settlement or payment-collection-time ledger. Low-stock alerts use an active-stock threshold of five or fewer units, not a forecast.

FORGE AI

Customer assistant. Local retrieval identifies products, categories, price limits, selected products and cart or approved order context. Product cards come from local records and remain available when optional specifications are missing. The assistant supports conversational context, explicit shopping preferences, comparisons and reviewed cart actions. Compatibility explanations identify missing facts rather than guaranteeing a fit. Local replies do not need a generation request; broader explanations use Gemini when configured and approved.

Find by image. After separate image-sharing approval, FORGE sends a resized reference image and up to 40 active catalog photos with public catalog facts to Gemini for visual comparison. Returned matches must refer to known catalog IDs and pass the implemented confidence threshold of 65. Cards use current local products. This is constrained catalog matching, not a general reverse-image search or a guarantee of exact model identification. The selected reference image is not saved to disk by FORGE.

Review summaries. An optional summary sends the product name, ratings and review text after review-summary approval. Reviewer identities and account details are excluded. A summary describes supplied reviews; it does not replace them or prove product quality.

AI Manager. Natural-language planning proposes typed operations: fix_content, find_low_stock, query_products, sales_summary and update_status. Java validates arguments and retrieves records. Content and status edits are previews until an administrator selects Apply reviewed changes. Existing nonempty content is preserved. Researching specifications is optional and requires exact-model evidence and source URLs; administrators still verify the variant and sources. Missing photographs require a real upload. No model-generated SQL is executed.

Limits and recovery. Shared transport distinguishes access problems, provider errors, timeouts and structured quota failures. Bounded retries and short-lived caches reduce repeated work. AI limits help reports provider restrictions without inventing remaining-use counters. Local inventory and commerce remain usable when generation is unavailable. The public feature name is FORGE AI; the privacy explanation identifies Google Gemini as the external provider.

6. OOP Implementation

Encapsulation

Product stores validated private final fields. Its constructor rejects blank names, invalid stock, invalid monetary values and oversized content. Photo arrays are copied on input and output so a caller cannot mutate stored object state through an array reference. ProductService replaces validated snapshots instead of exposing unrestricted setters.

Cart keeps its item map private. getItems returns an unmodifiable list copy; callers use add, setQuantity, remove or clear. A proposed cart change is persisted before the visible map is replaced. Order and related records copy owned lists. Records generate private final component fields and public component accessors; record notation does not make the fields public.

Inheritance

User is an abstract parent with private identity fields and public identity getters. Customer, Admin and Guest extend it and override canManageProducts, canShop and getRole. Administrators can maintain products, customers can purchase, and guests can browse and build a cart while checkout still requires a Customer. Subclasses inherit the identity behavior while the parent fields remain private.

OutOfStockException also extends ValidationException, allowing the UI to handle a specialized stock failure as a user-correctable validation problem. The role hierarchy is the primary business example; JavaFX inheritance alone is not the evidence used for this requirement.

Polymorphism

CheckoutService receives a PaymentMethod reference and executes payment.pay(total). At runtime, CashOnDeliveryPayment accepts the order with PENDING payment, while SimulatedWalletPayment returns PAID for approval or rejects a declined payment. Checkout does not select the concrete payment implementation with a type-based branch.

Service calls such as actor.canManageProducts also dispatch through the User abstraction to the current role's override. A Customer and an Admin therefore give different permission results through the same method contract.

```
// Excerpt from CheckoutService.checkoutLocked
PaymentResult result = payment.pay(total);
if (!result.accepted())
    throw new ValidationException(
        "Demo wallet payment declined. Your cart and stock are unchanged.");
```

Abstraction

Abstract User defines the role behavior every concrete user must provide. PaymentMethod defines getName and pay without exposing how a payment simulation chooses its result. Checkout depends on that contract. A new local payment simulation can implement the interface without rewriting the checkout transaction.

Services and repositories also separate responsibilities: a view asks for a checkout operation rather than executing its SQL. This architectural separation supports the explicit Java abstract class and interface examples; it is not a substitute for them.

Transaction and class relationships

Cart owns its CartItem collection, while each CartItem refers to an independent Product snapshot. Order owns purchase-time OrderItems and fulfillment events. Customer-to-order ownership is stored by customerId; the record does not contain a Customer object field. UML distinguishes direct field associations, composition, inheritance, interface realization and call dependencies.

Checkout follows this order: validate customer and cart ownership; open one database connection; disable autocommit; reload products; reject stale prices; conditionally deduct stock; invoke PaymentMethod; save order snapshots; delete saved-cart rows; commit; then clear the visible cart. An exception rolls back the database changes and preserves the cart. Real payment side effects would require a separate recovery design; these implementations are local simulations.

AI Manager follows prepare, review and apply. AiManagerService prepares a Result; ManagerAi proposes validated tool calls and drafts; ProductService.applyManagerChanges enforces administrator access, validates the expected product snapshots, writes one reviewed batch and records before and after values. Stale edits roll back the entire batch, and an already-applied batch cannot run twice.

Important code locations

Responsibility	Java source under src/main/java/edu/course/shop
Entry point and wiring	ShopApp.java and Services.java
Role inheritance and abstraction	model/User.java, Customer.java, Admin.java and Guest.java
Encapsulated product and cart state	model/Product.java and model/Cart.java
Payment contract and overrides	payment/PaymentMethod.java and both implementations
Atomic checkout	service/CheckoutService.java
Saved carts and fulfillment	service/PersistentCartService.java and service/OrderService.java
Local assistant retrieval	service/CatalogAssistant.java
AI generation and image matching	service/GeminiAssistant.java and service/AiHttp.java
Reviewed administrative AI changes	service/AiManagerService.java, ManagerAi.java and ProductService.java
JDBC schema and queries	persistence/DatabaseManager.java and repository classes

7. UML Class Diagram

The following 15 sheets describe the same implementation from different viewpoints. Sheets 1 to 14 include every one of the 104 top-level Java types. Sheet 15 shows relationships across application layers. The source snapshot also contains 61 named nested types, for a total of 165; their complete declared members are provided in the companion class reference.

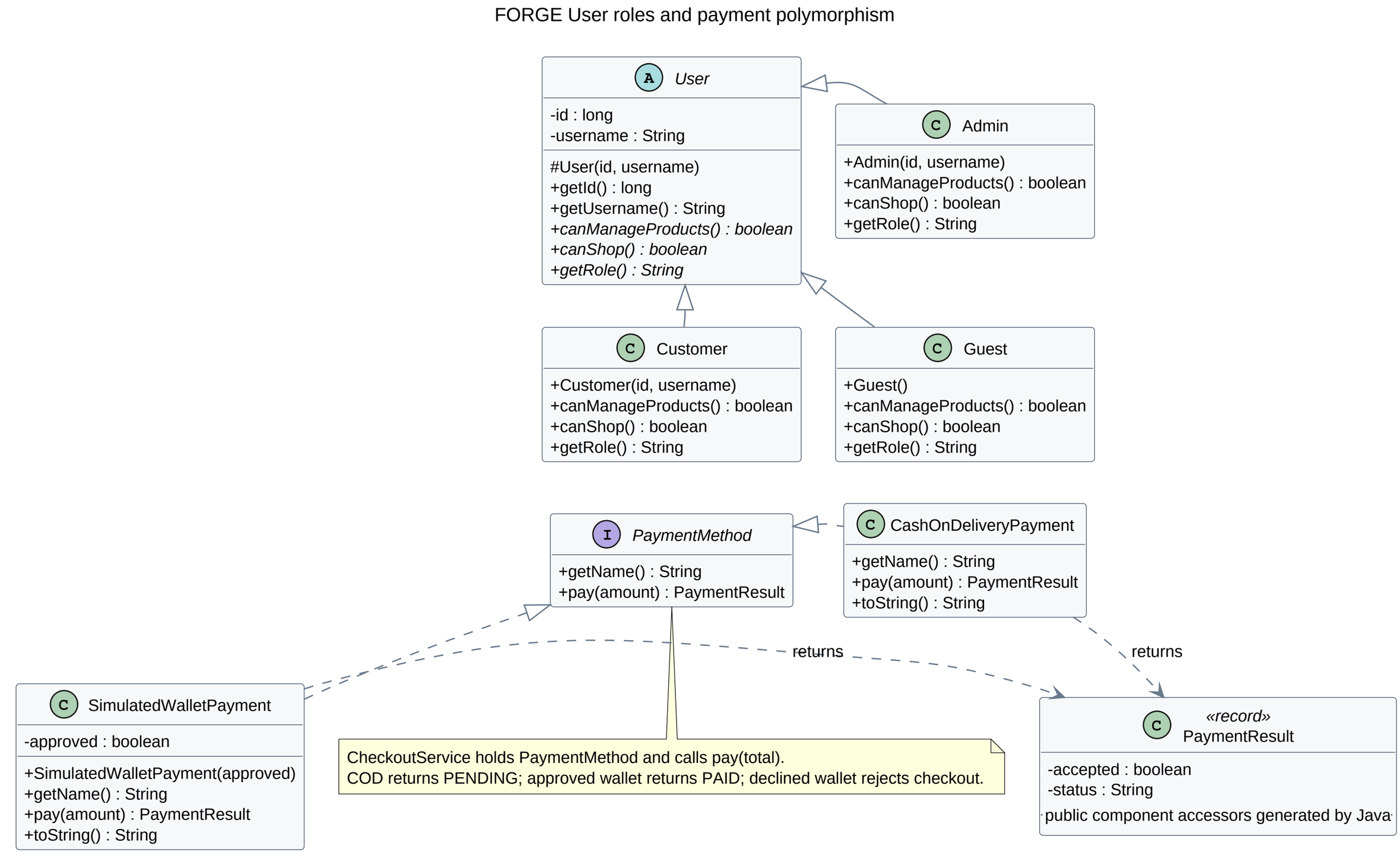
Notation: + means public, # protected, - private and ~ package access. Solid generalization arrows show extends; dashed realization arrows show implements. Filled diamonds indicate owned collections; ordinary associations indicate stored references. Dashed dependency arrows indicate construction or calls. Hidden layout links are not semantic relationships.

Readable sheets select API operations and omit repeated overloads or long parameter lists. Record components are shown with their private storage, with a note about generated public accessors. Exact field types, full parameter types, overloads, visibility, source lines, inheritance and all explicitly declared methods are retained in uml/CLASS-REFERENCE.md, uml/source-inventory.json and uml/complete-members.puml. The inventory excludes anonymous classes and compiler-generated members rather than pretending they were declared in source.

The diagrams are exported as SVG for zooming and as a separate FORGE-UML.pdf. The source is docs/classes.puml, with individual sheets under docs/uml. Recheck the source hashes and regenerate the inventory whenever Java classes change.

UML sheet 1 User roles and payment polymorphism

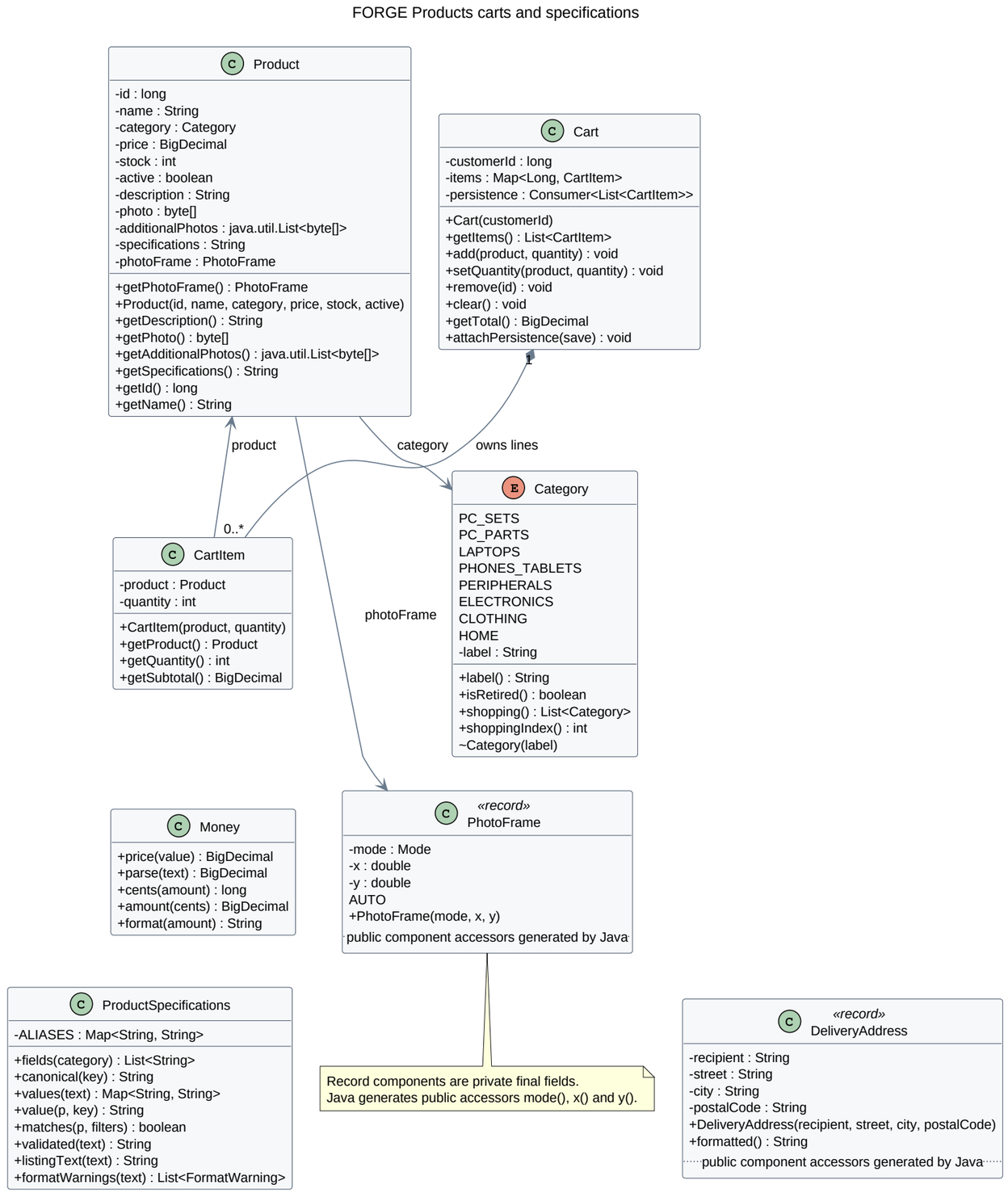
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 2 Products carts and specifications

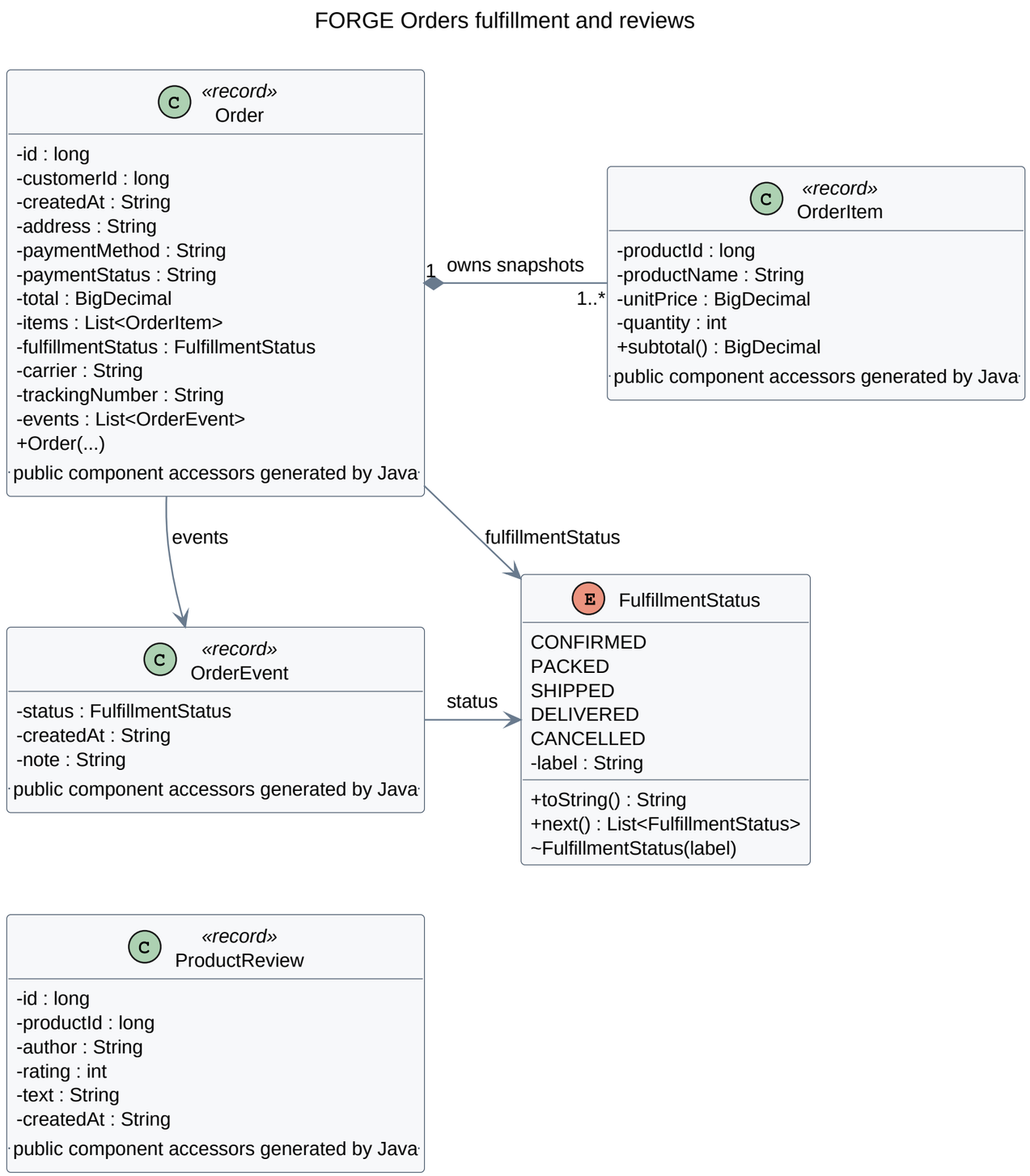
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 3 Orders fulfillment and reviews

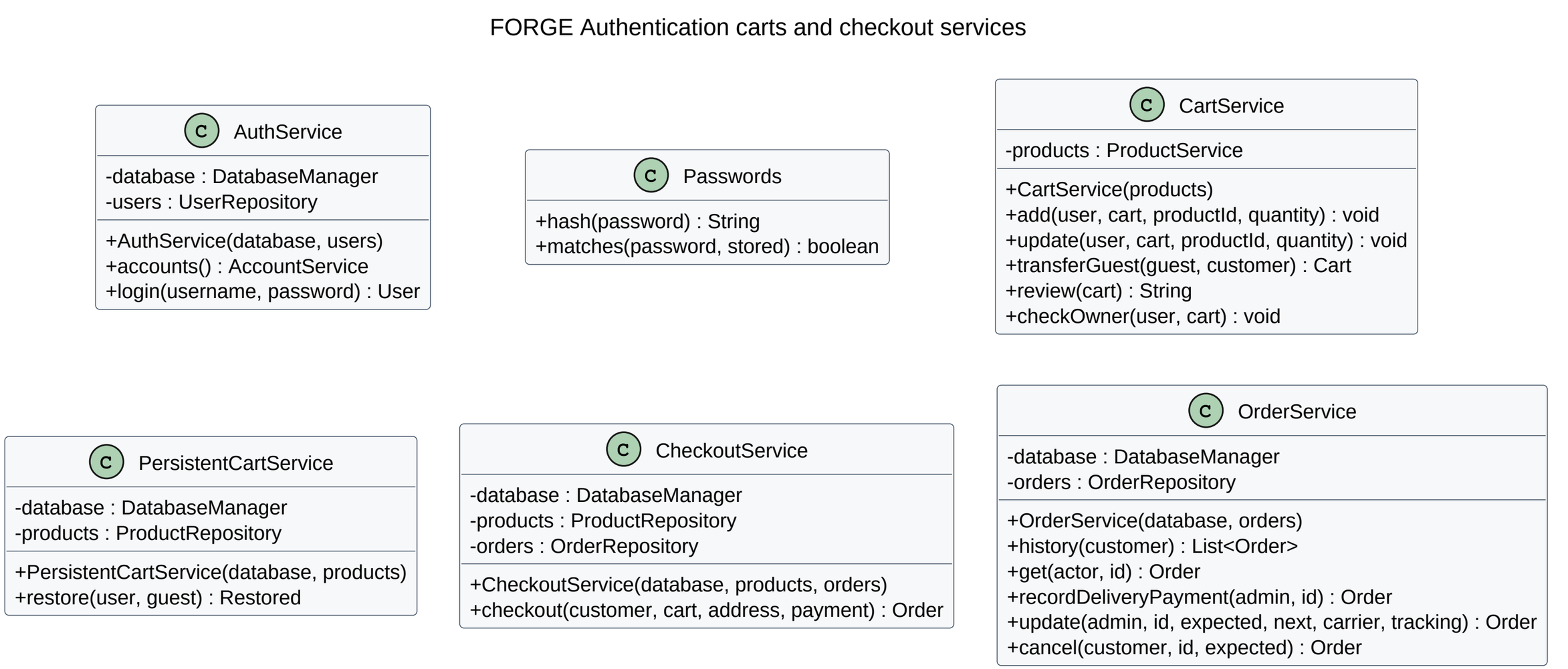
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 4 Authentication carts and checkout services

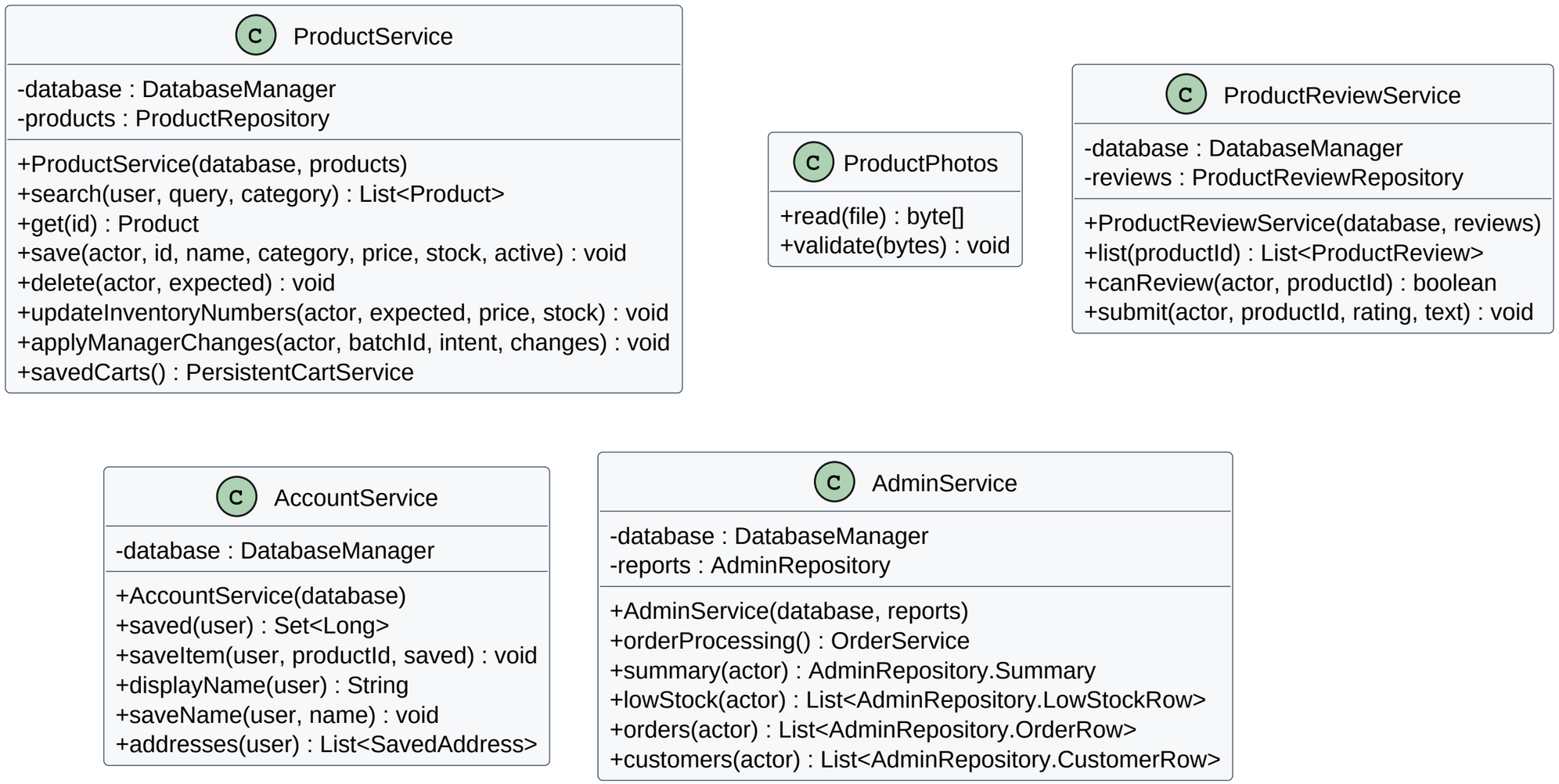
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

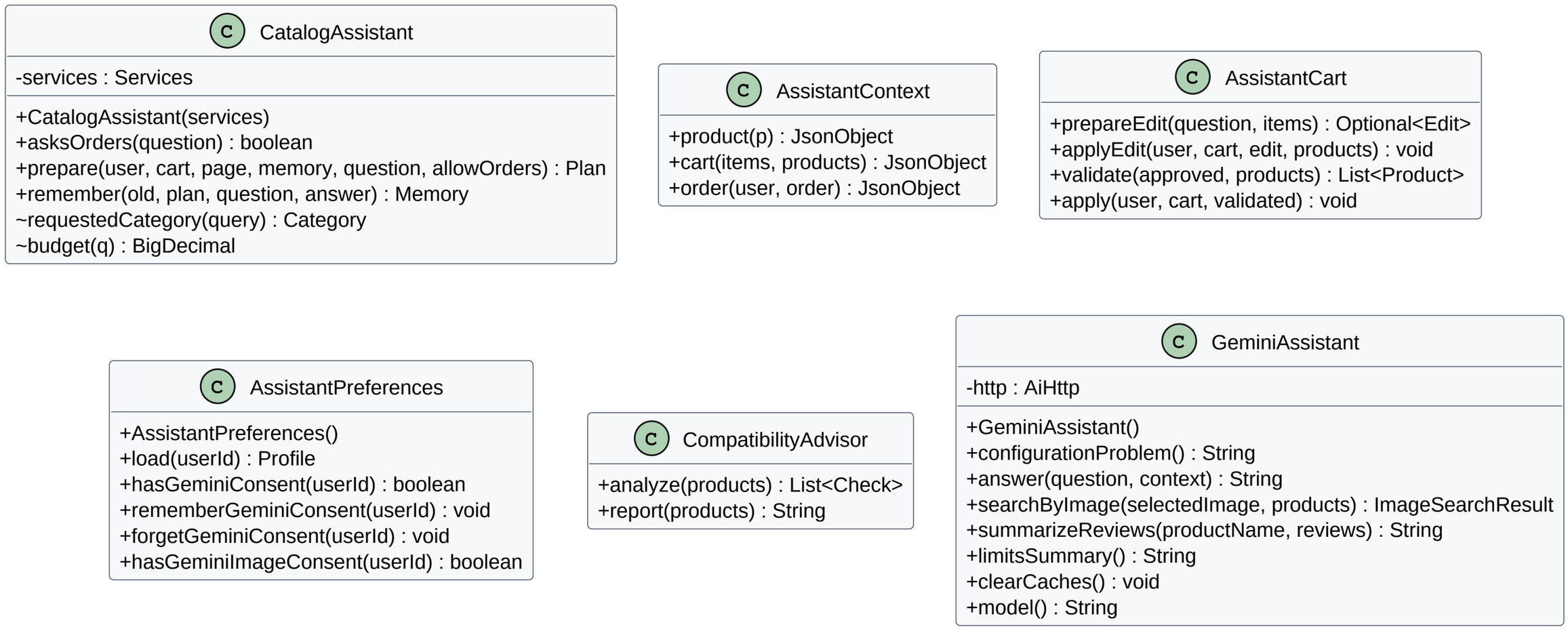
UML sheet 5 Inventory accounts and administration services

Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



UML sheet 6 Customer assistant retrieval and generation

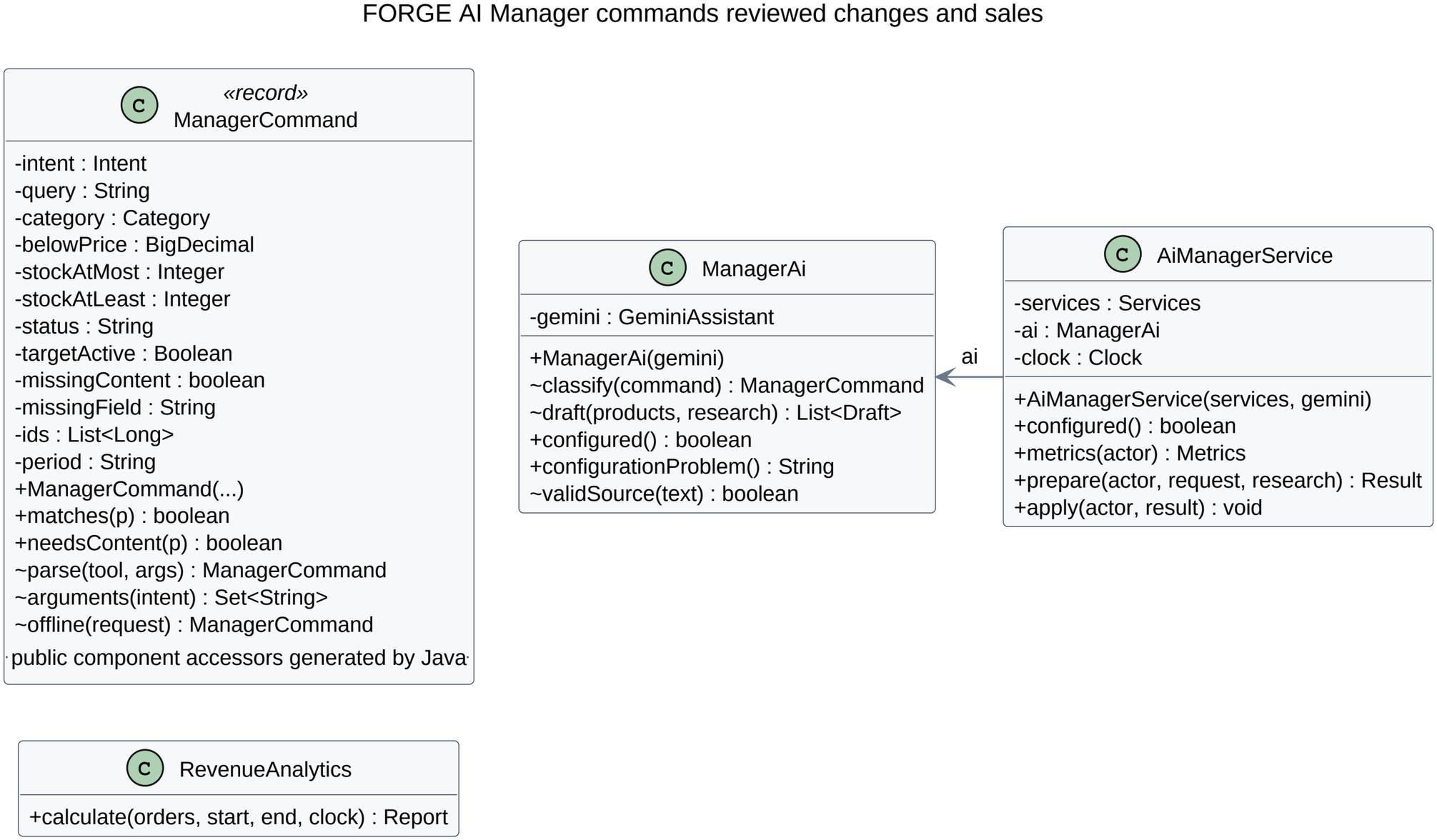
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 7 AI Manager commands reviewed changes and sales

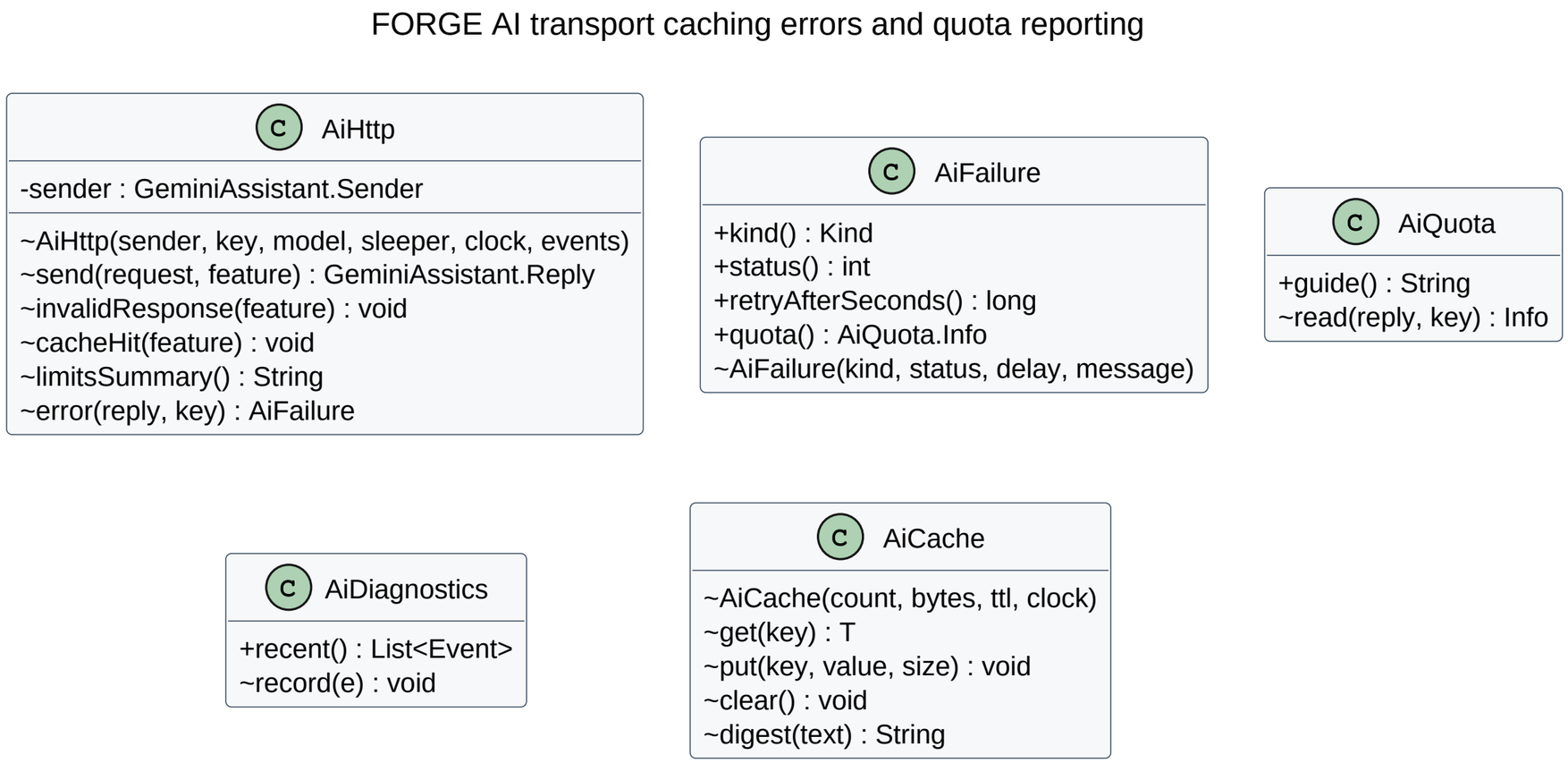
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 8 AI transport caching errors and quota reporting

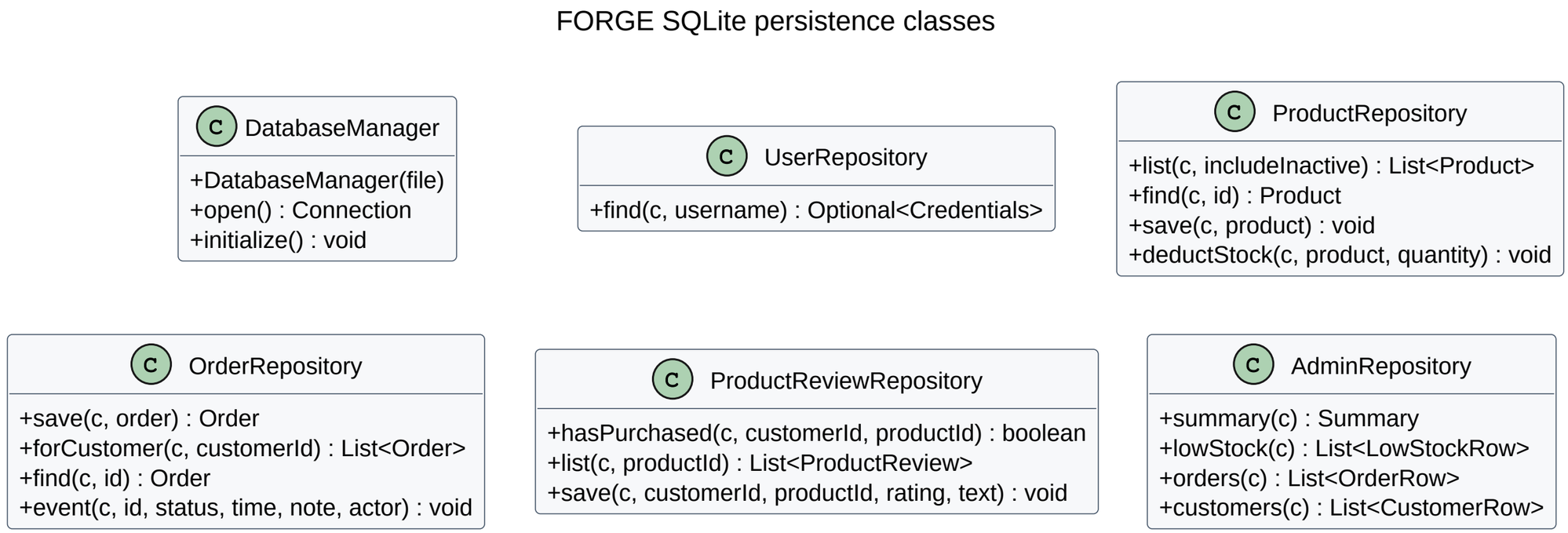
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

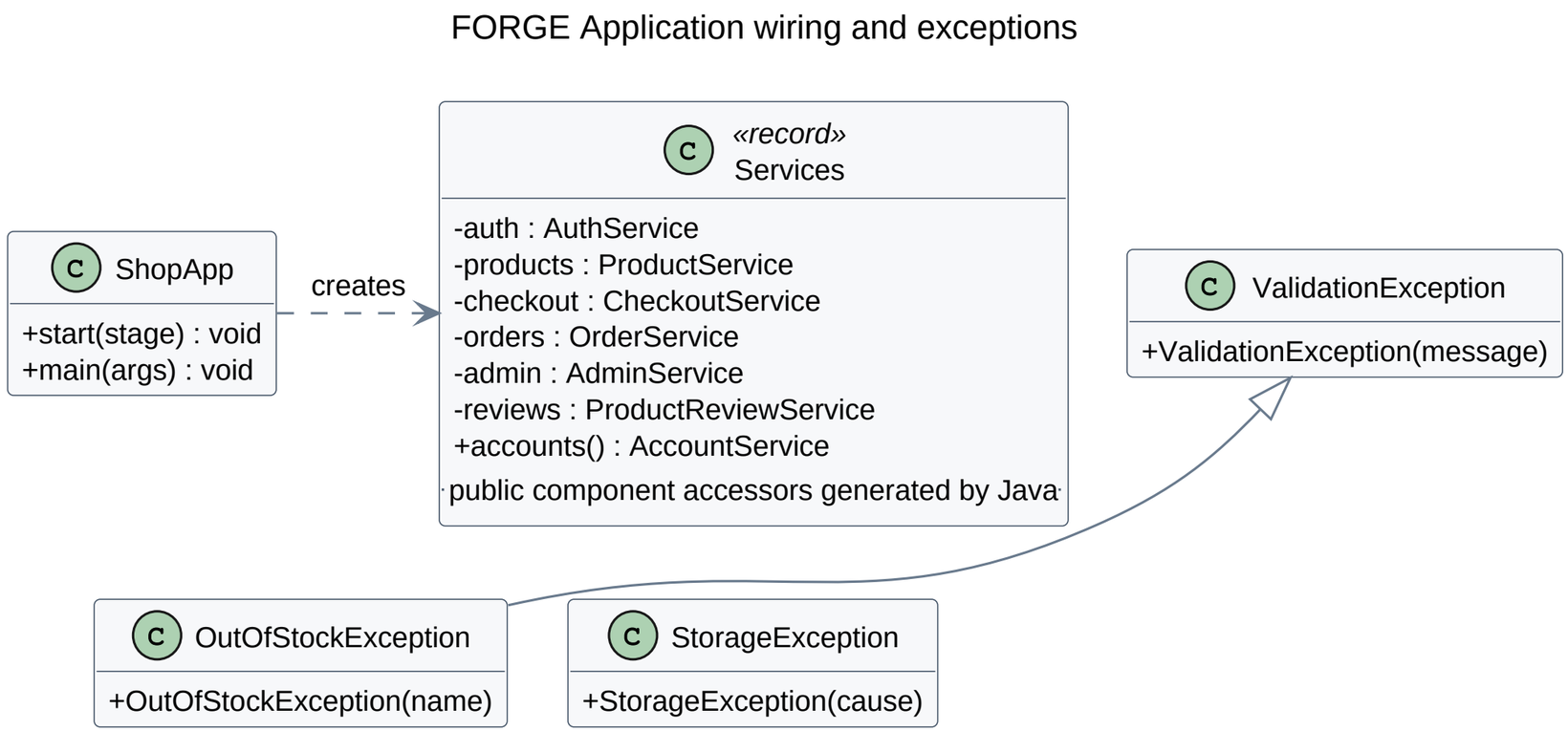
UML sheet 9 SQLite persistence classes

Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



UML sheet 10 Application wiring and exceptions

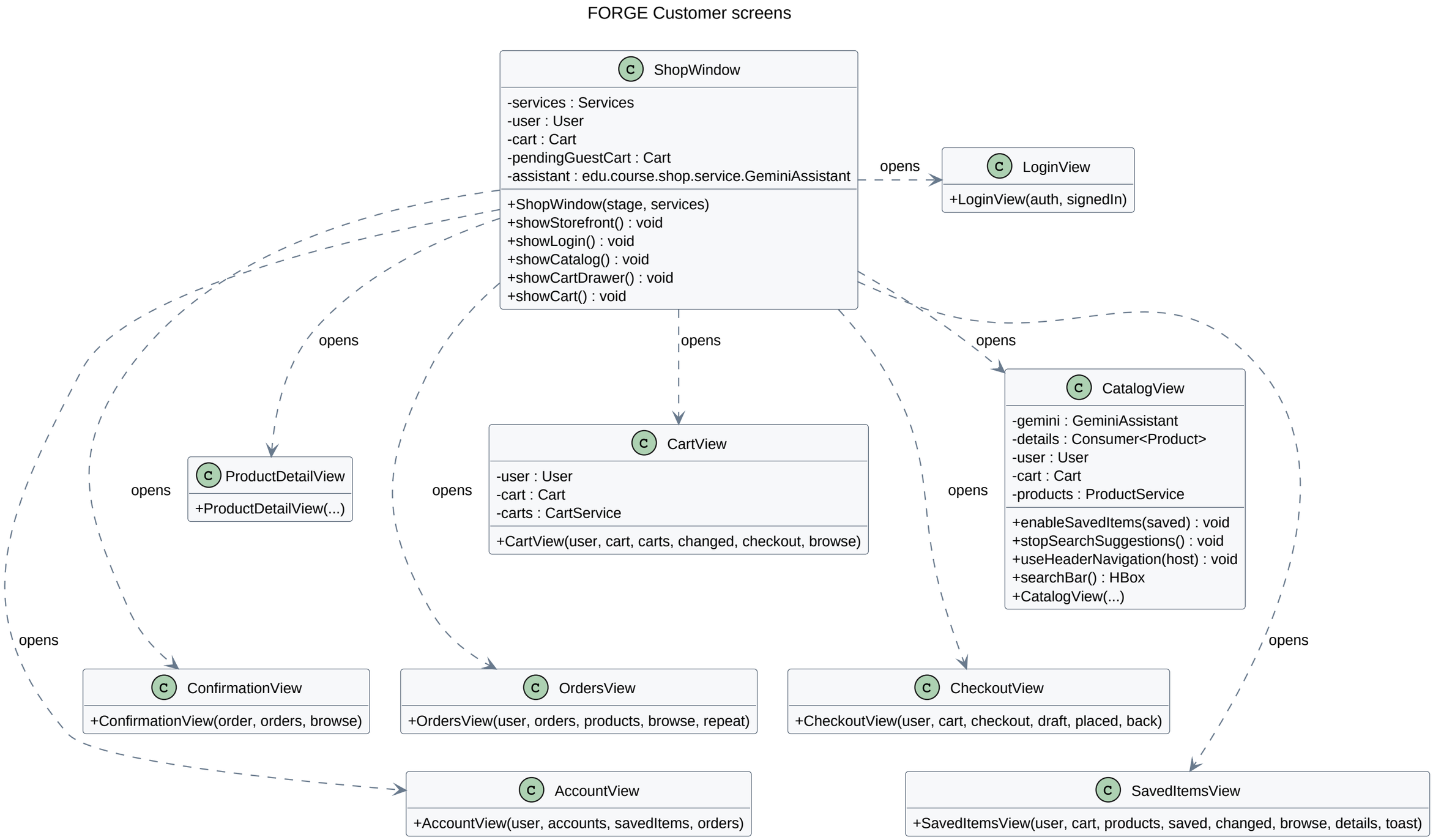
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 11 Customer screens

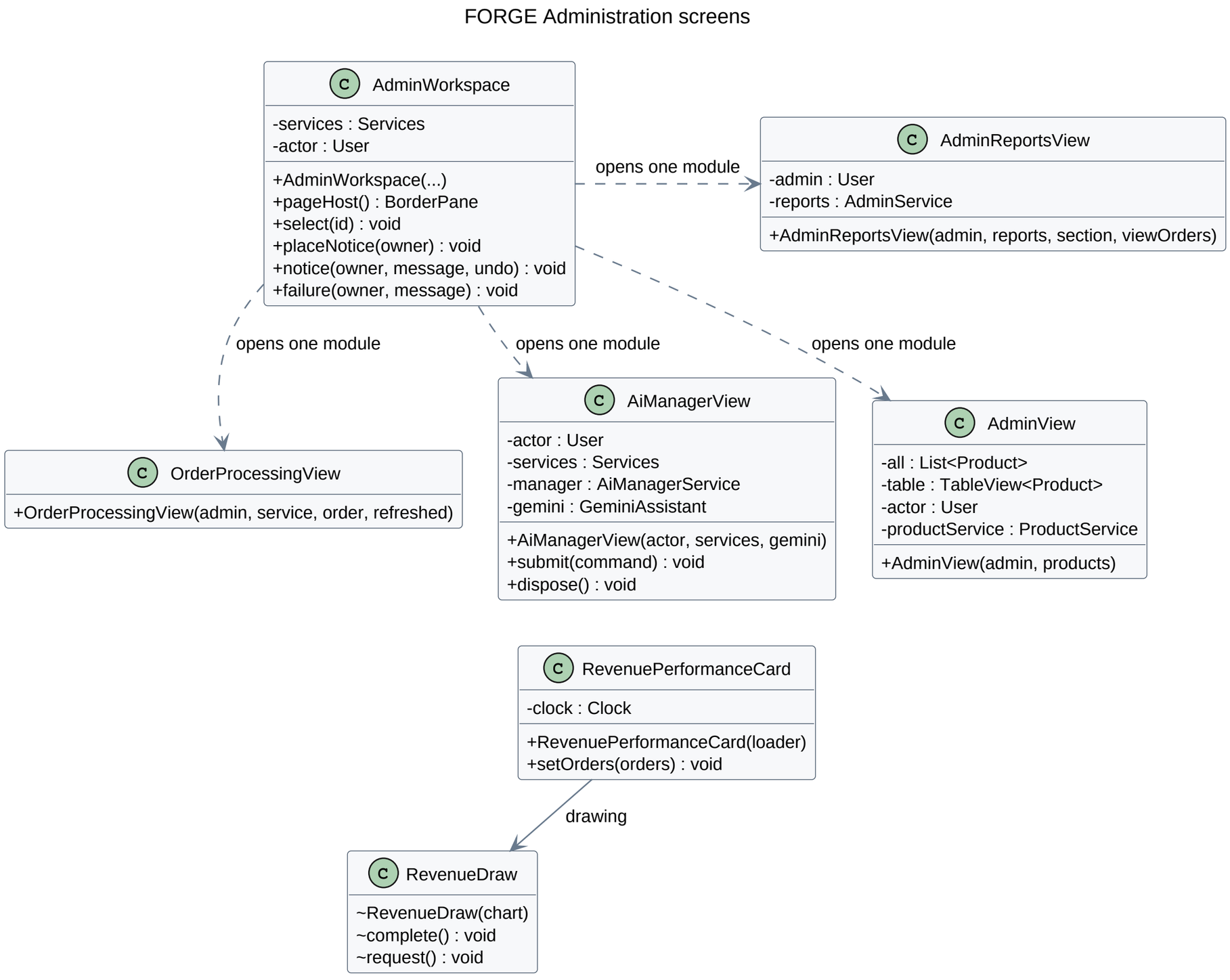
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 12 Administration screens

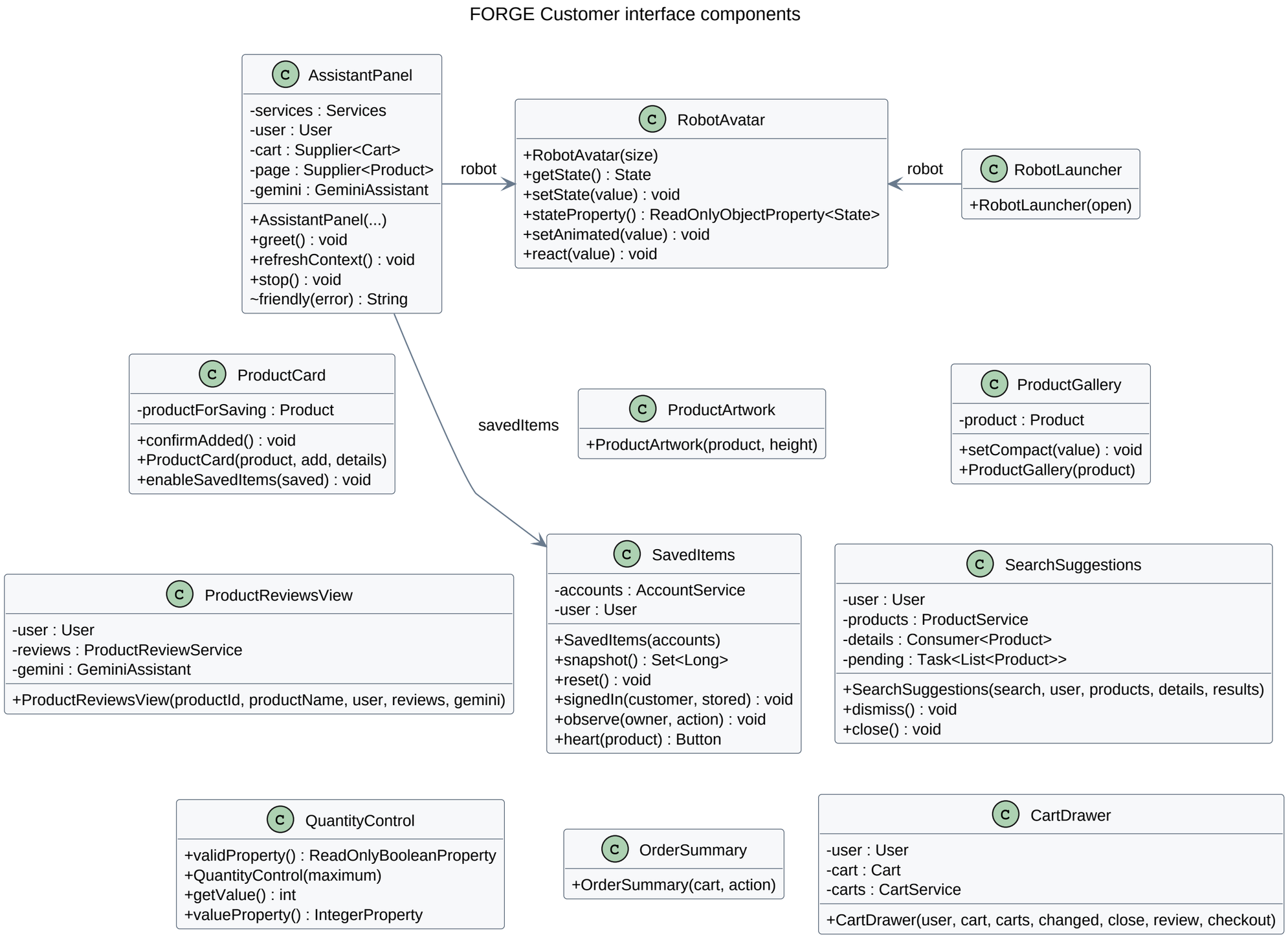
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 13 Customer interface components

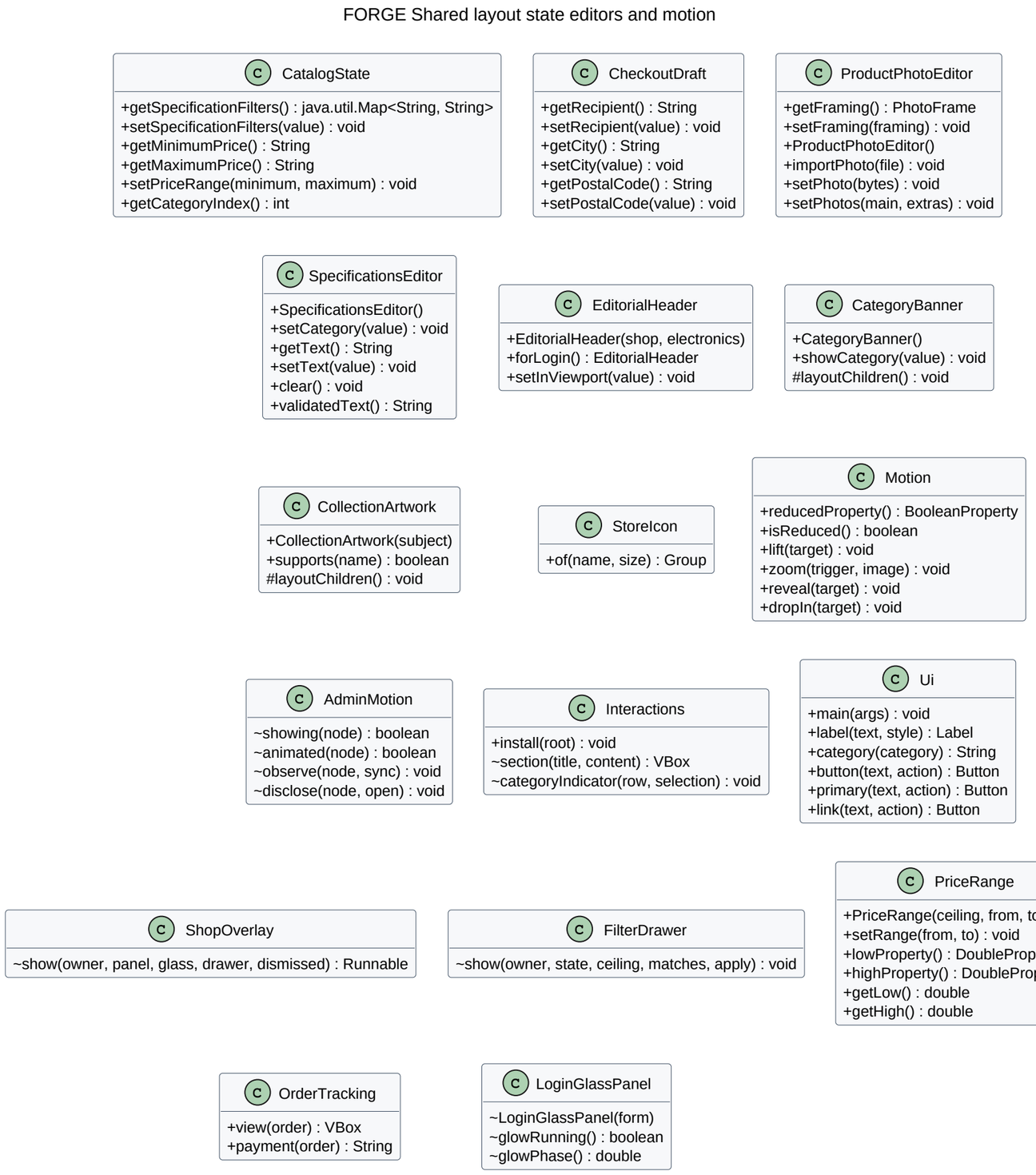
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 14 Shared layout state editors and motion

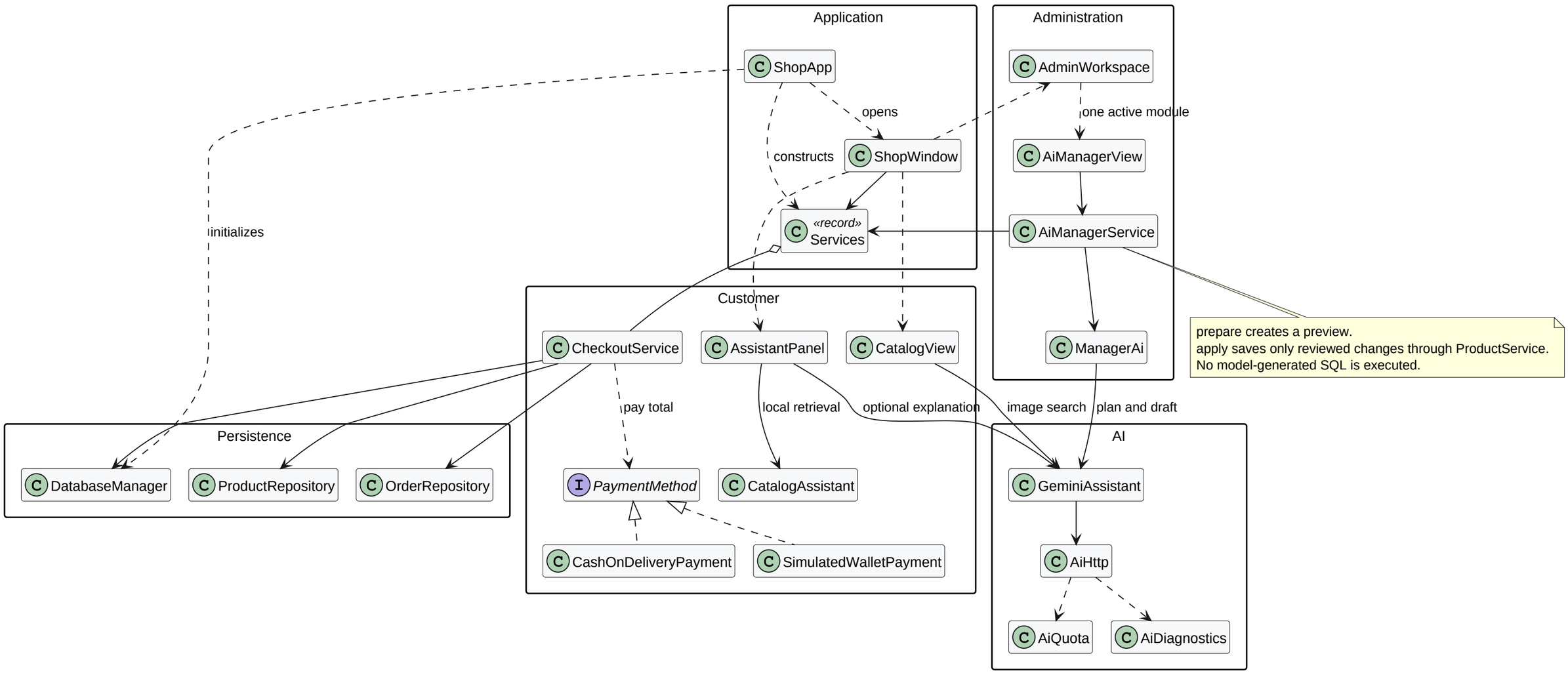
Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



Declared API excerpts. Full signatures and nested types are in CLASS-REFERENCE.md.

UML sheet 15 Dependencies across application layers

Section 7 | FORGE class diagram | Declared API excerpts; full signatures in the companion class reference



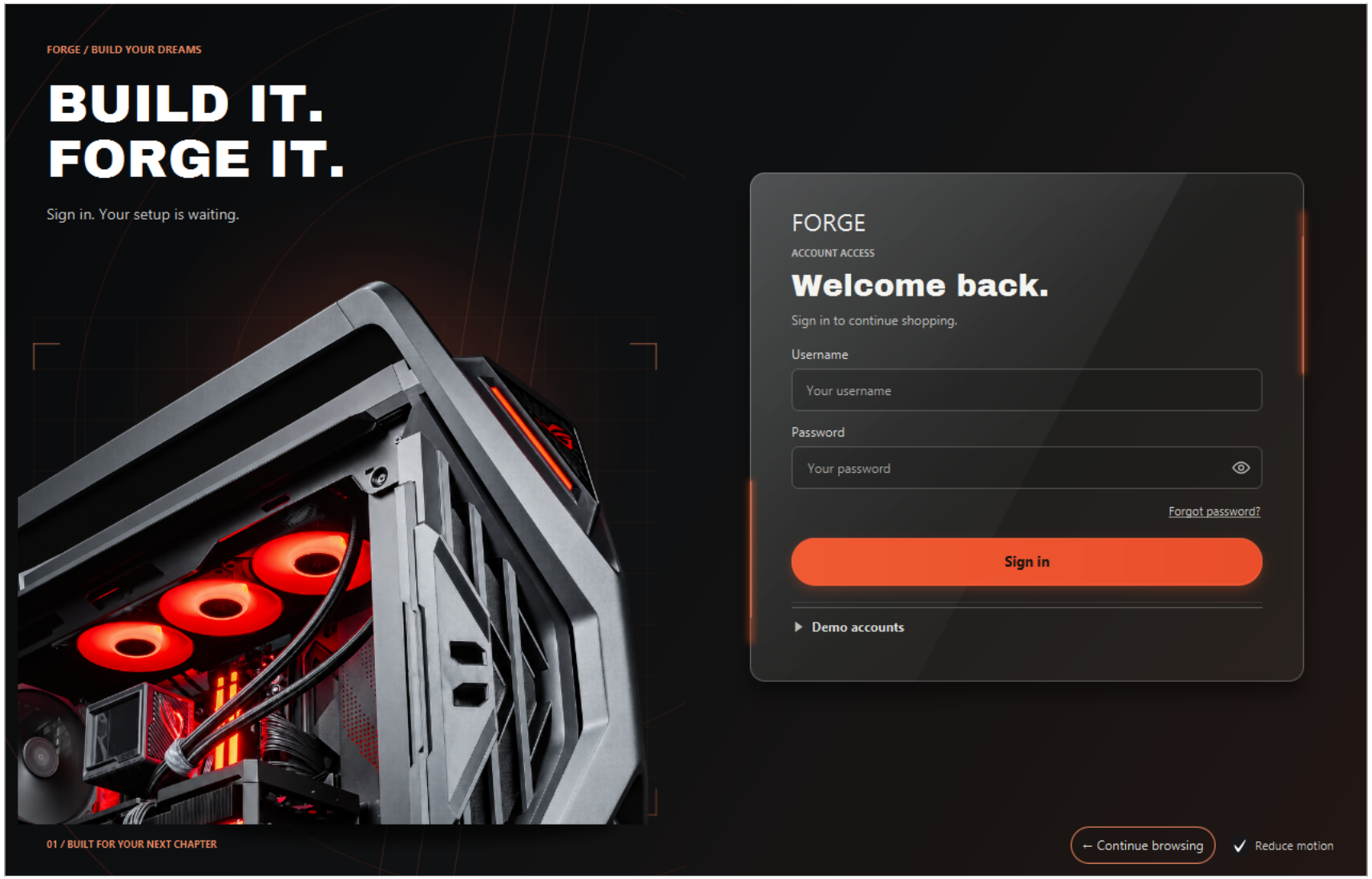
8. GUI Screenshots

The following screenshots were captured on 24 October 2026 from actual JavaFX controls using the current compiled source. They use fresh isolated SQLite databases and synthetic catalog and order records. AI Manager previews use deterministic provider fixtures, allowing the UI and approval flow to be tested without sending a live request. They are application scene snapshots, not design mockups or evidence of a live provider's availability.

Screenshots cover account access, customer shopping, product detail, cart, checkout, order tracking, admin overview, inventory, the specification editor, order processing, customer product cards and AI Manager review. Captions identify the corresponding verification suite and the feature demonstrated.

Figure 1 Account access

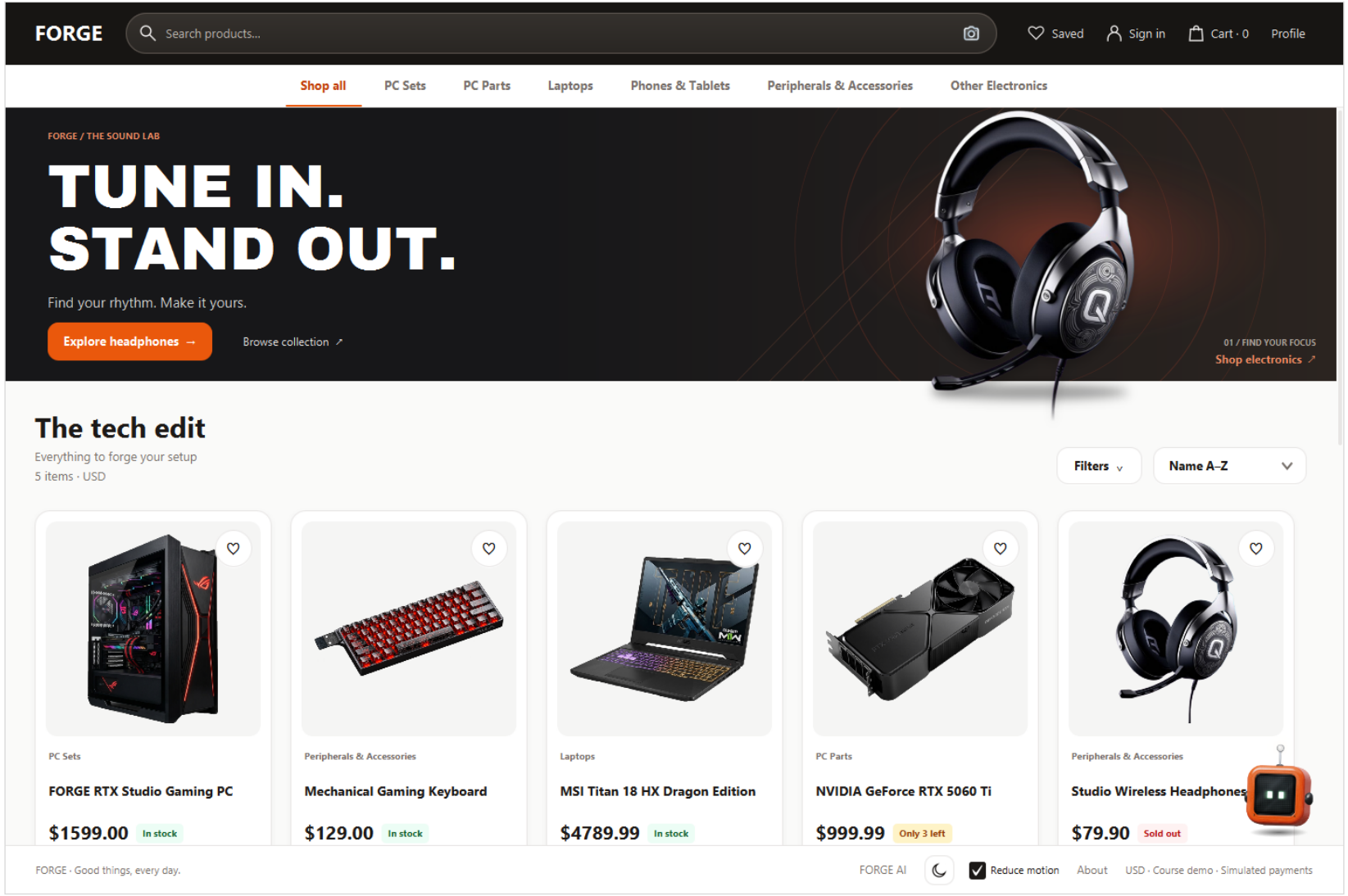
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



The glass login panel provides required-field validation, password visibility, public demo account help and a reduced-motion setting. Verification: LoginGlassSmoke.

Figure 2 Customer storefront

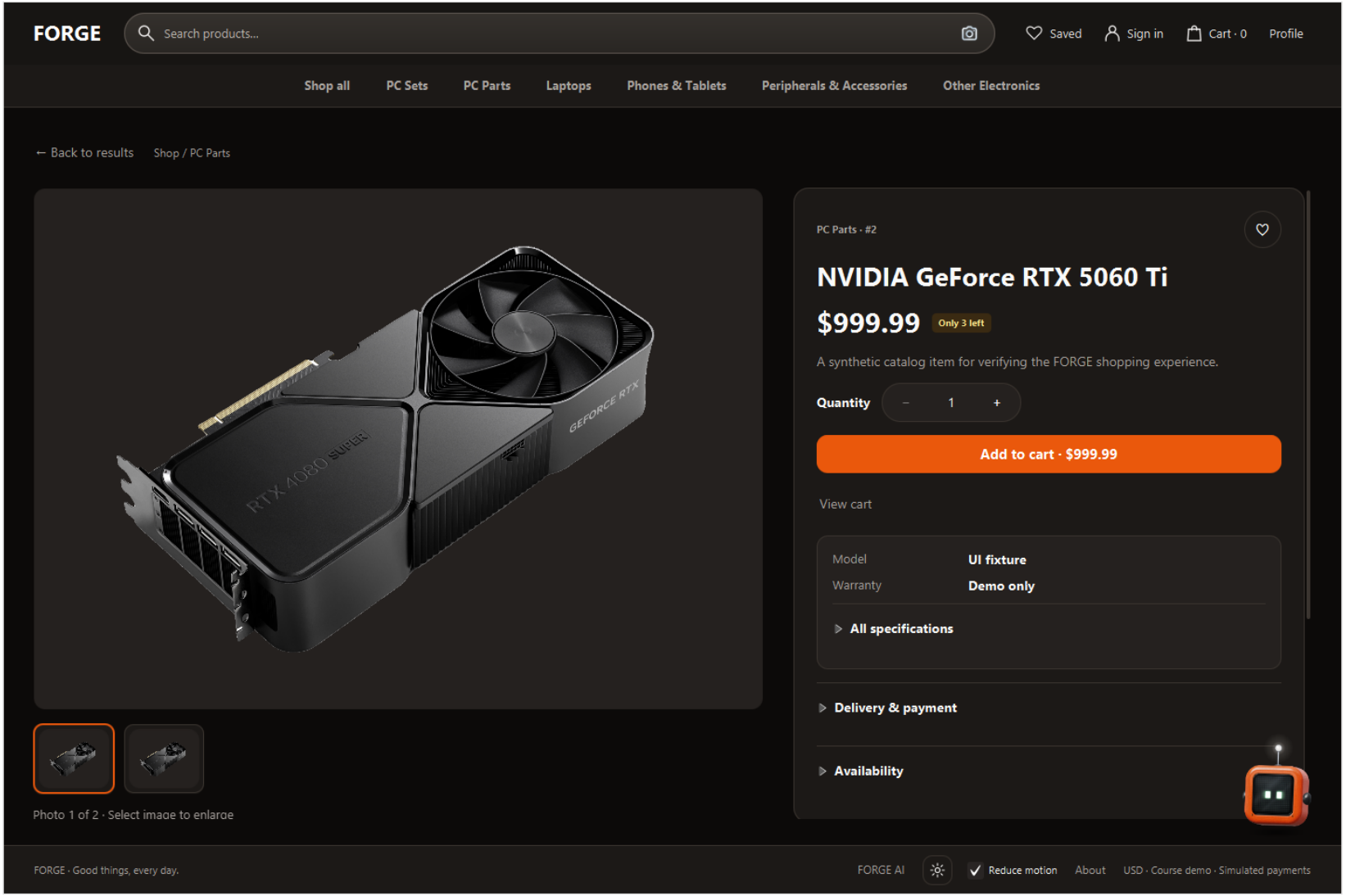
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



The store separates category navigation from the search and account controls. Product cards show current fixture prices and availability. Verification: StorefrontUiSmoke.

Figure 3 Product details

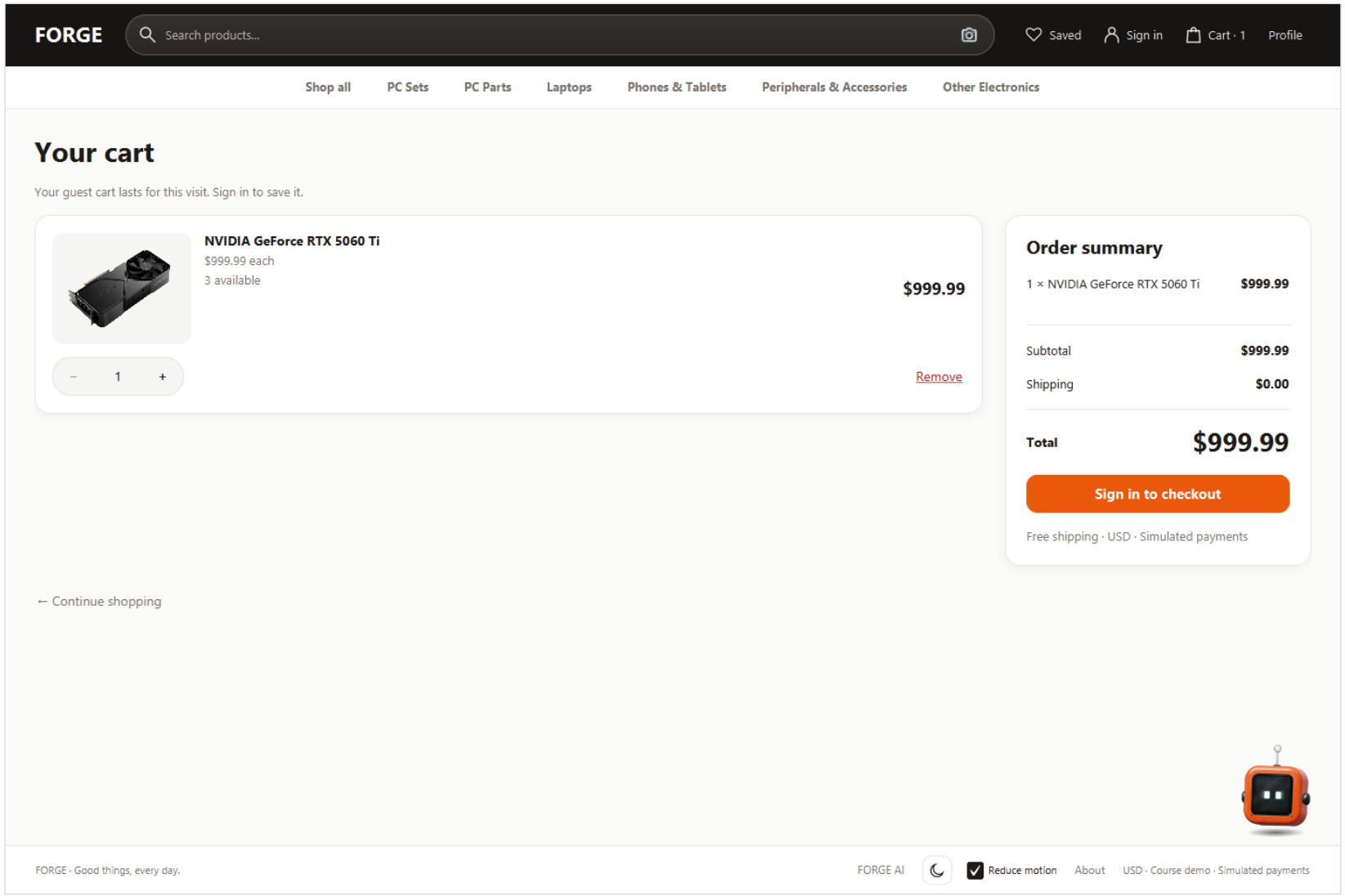
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



The detail interface presents a product image, optional specifications and purchase controls. Missing specifications do not hide the product. Verification: StorefrontUiSmoke.

Figure 4 Shopping cart

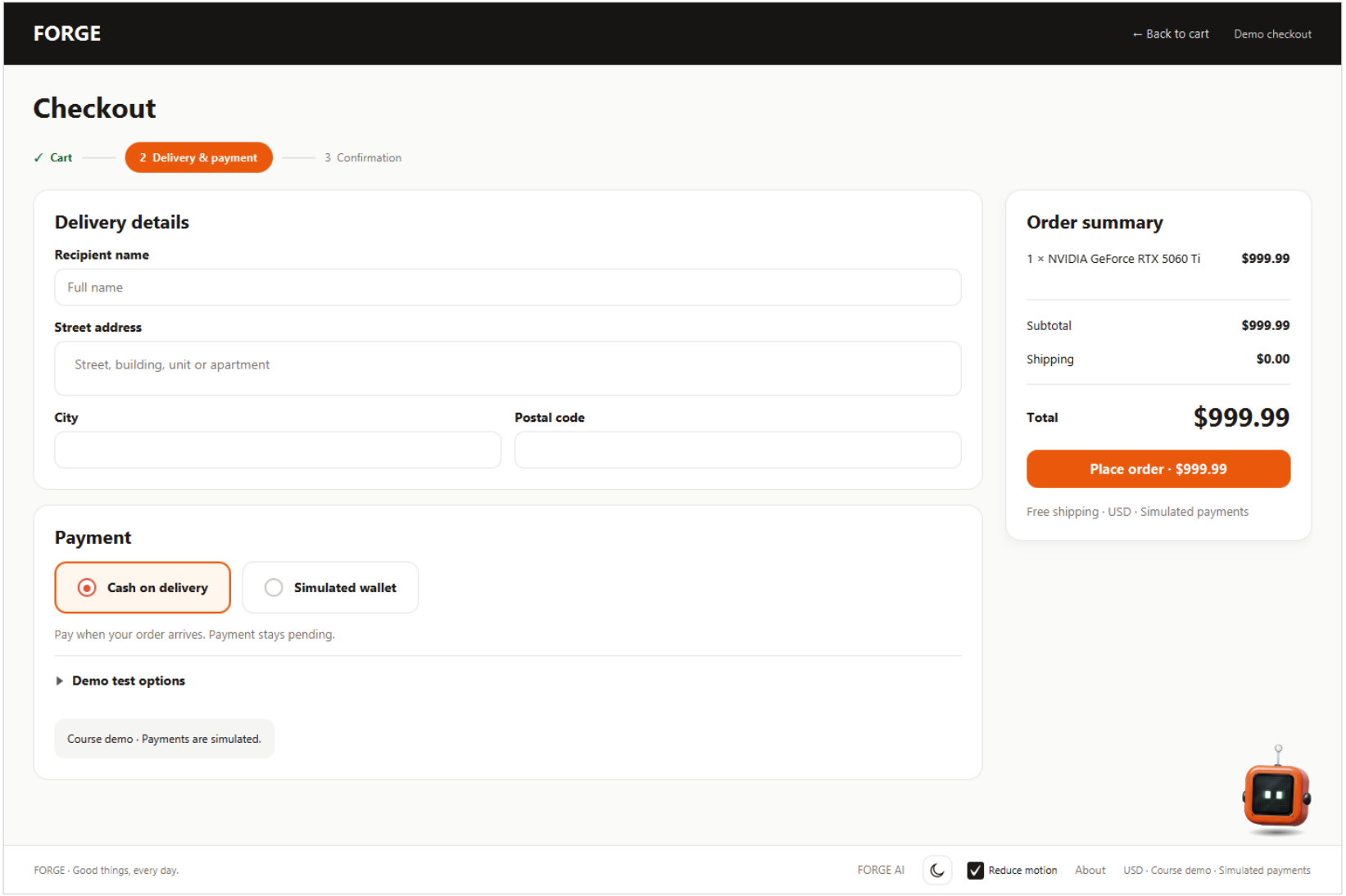
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



The cart exposes quantities, removal controls and a shared order summary. A cart does not reserve stock. Verification: StorefrontUiSmoke.

Figure 5 Checkout

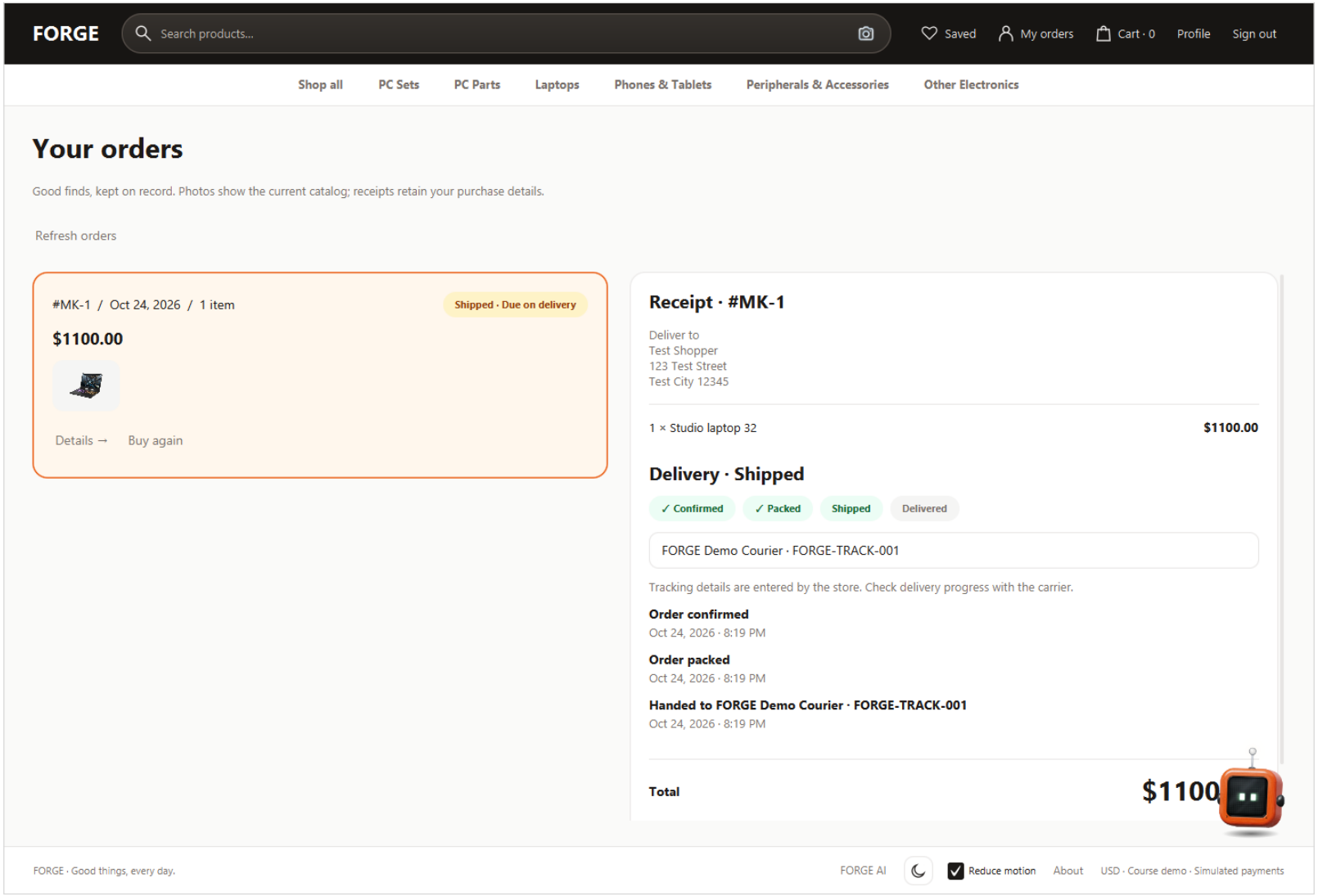
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



Checkout collects validated delivery details and a simulated payment choice. The service rechecks prices and stock before committing. Verification: StorefrontUiSmoke.

Figure 6 Customer order history and tracking

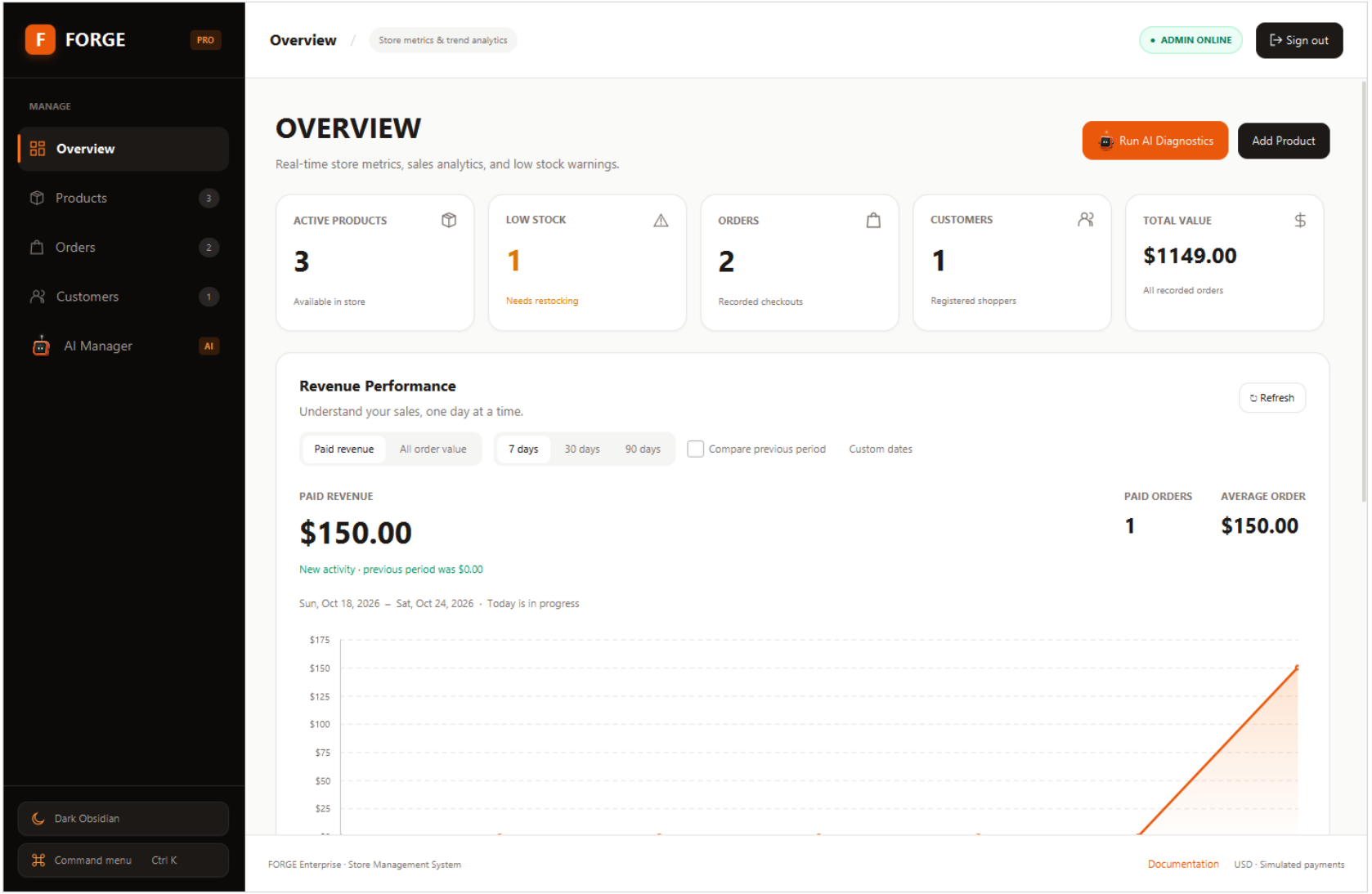
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



Customers see their own recorded fulfillment timeline. Carrier and tracking details are entered by an administrator; no live carrier API is connected. Verification: CommerceUiSmoke.

Figure 7 Administration overview

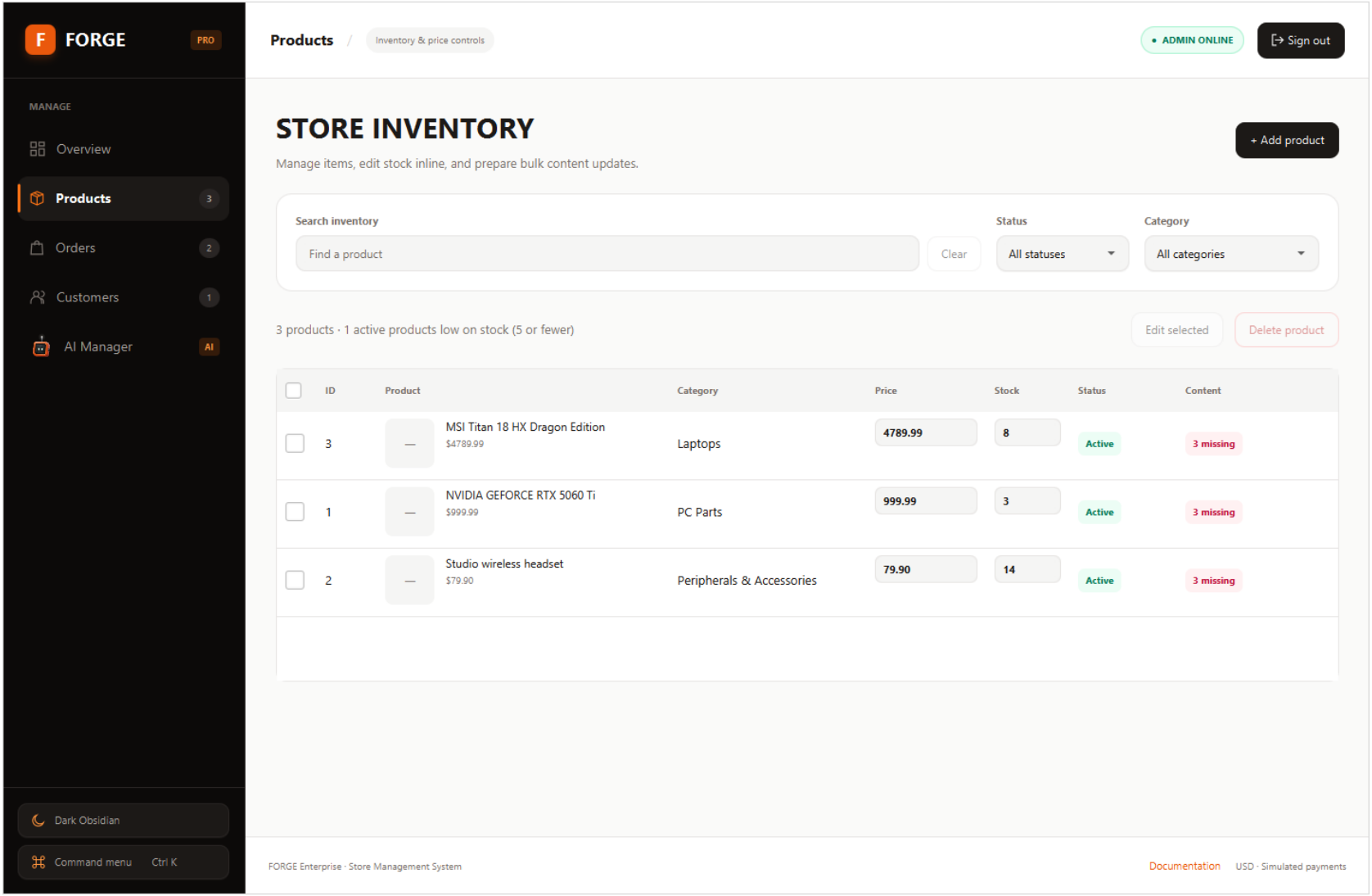
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



Overview uses SQLite order records for metrics and the revenue graph. These values are synthetic verification data, not actual business income. Verification: AdminDashboardUiSmoke.

Figure 8 Inventory management

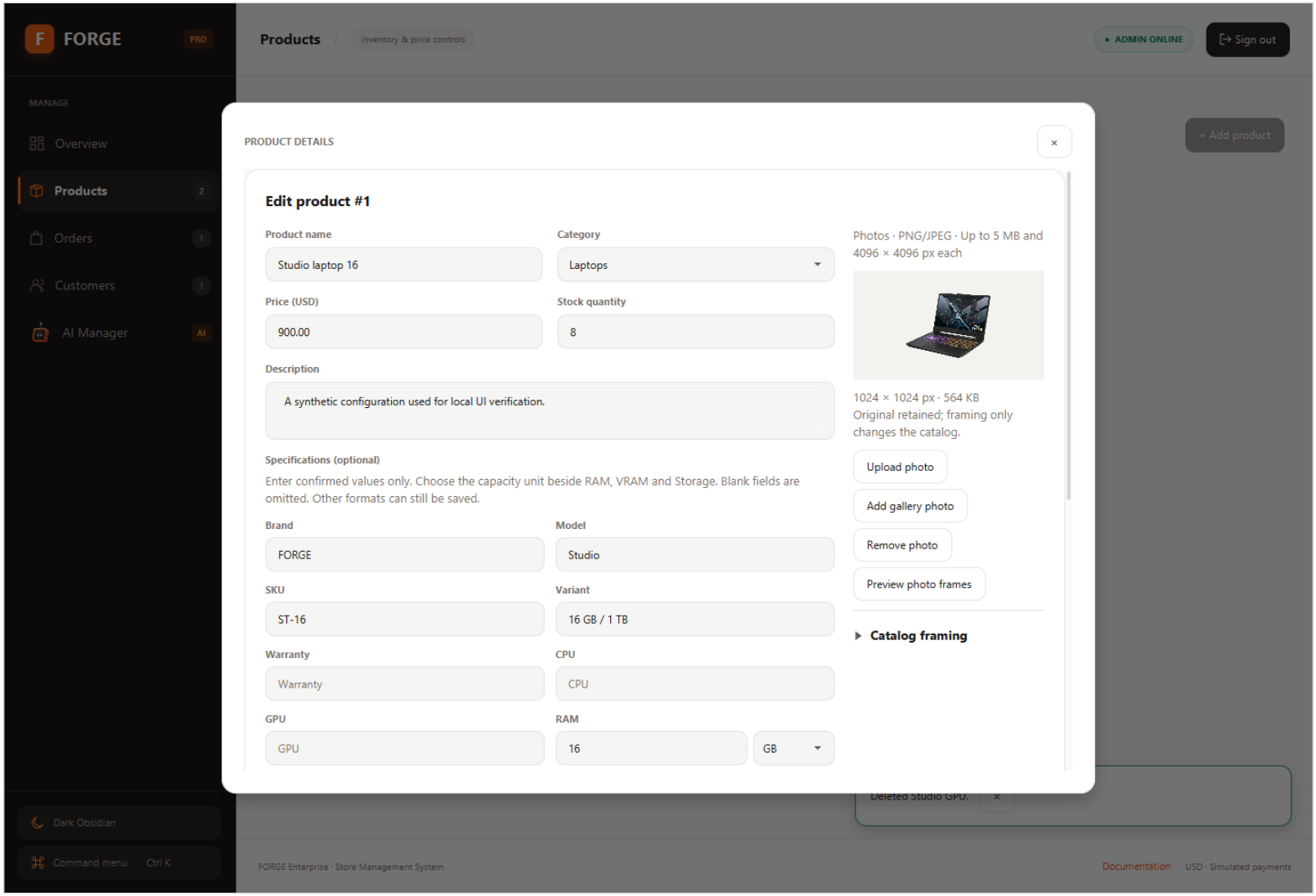
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



The inventory table supports searching, filtering and validated inline stock or price edits. Product and content badges remain readable. Verification: AdminDashboardUiSmoke.

Figure 9 Product editor and specifications

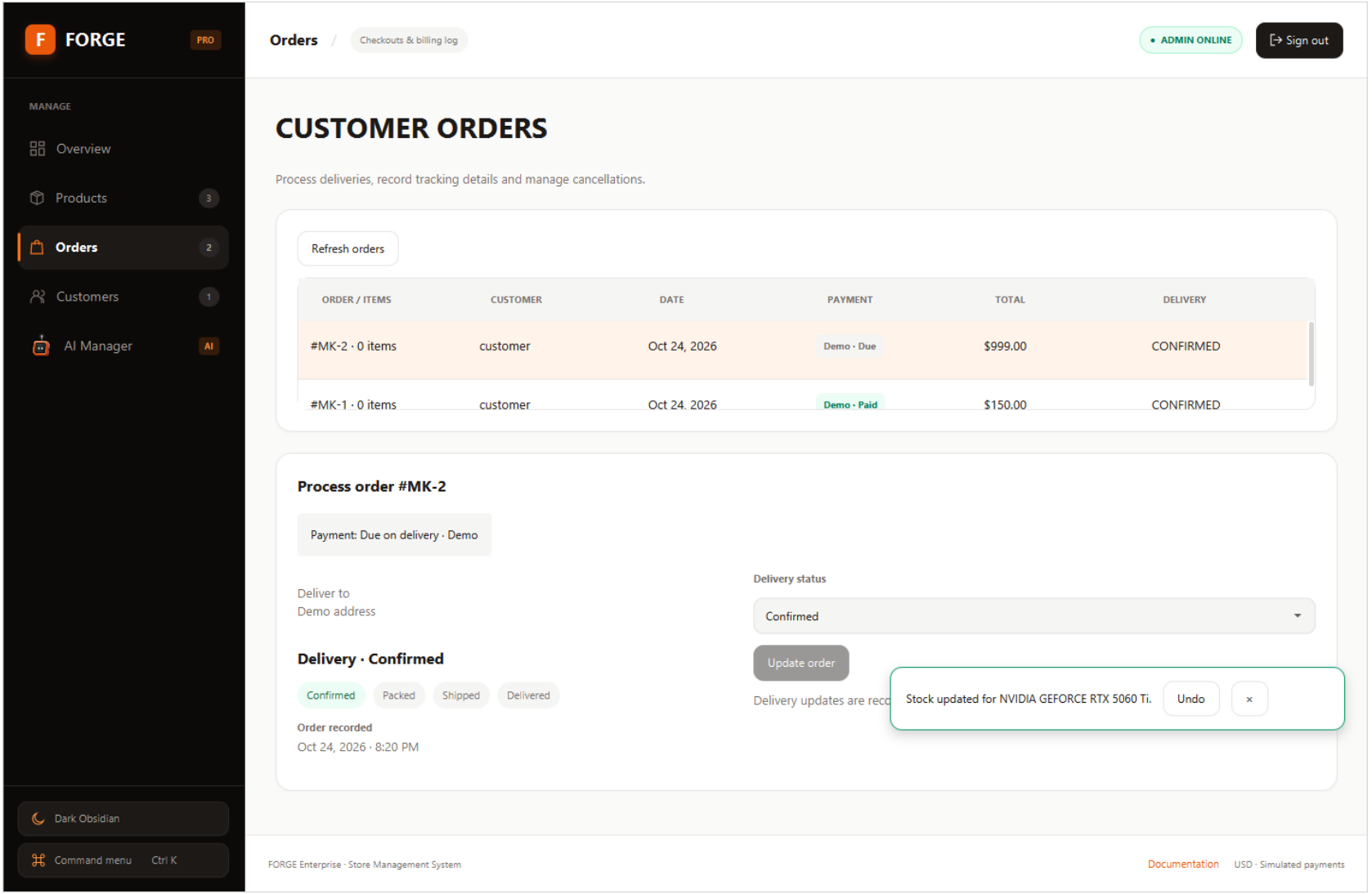
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



The editor supports category-specific facts and an optional free-text specification format. Quantities use explicit units and photos are supplied by the administrator. Verification: CommerceUiSmoke.

Figure 10 Administration order processing

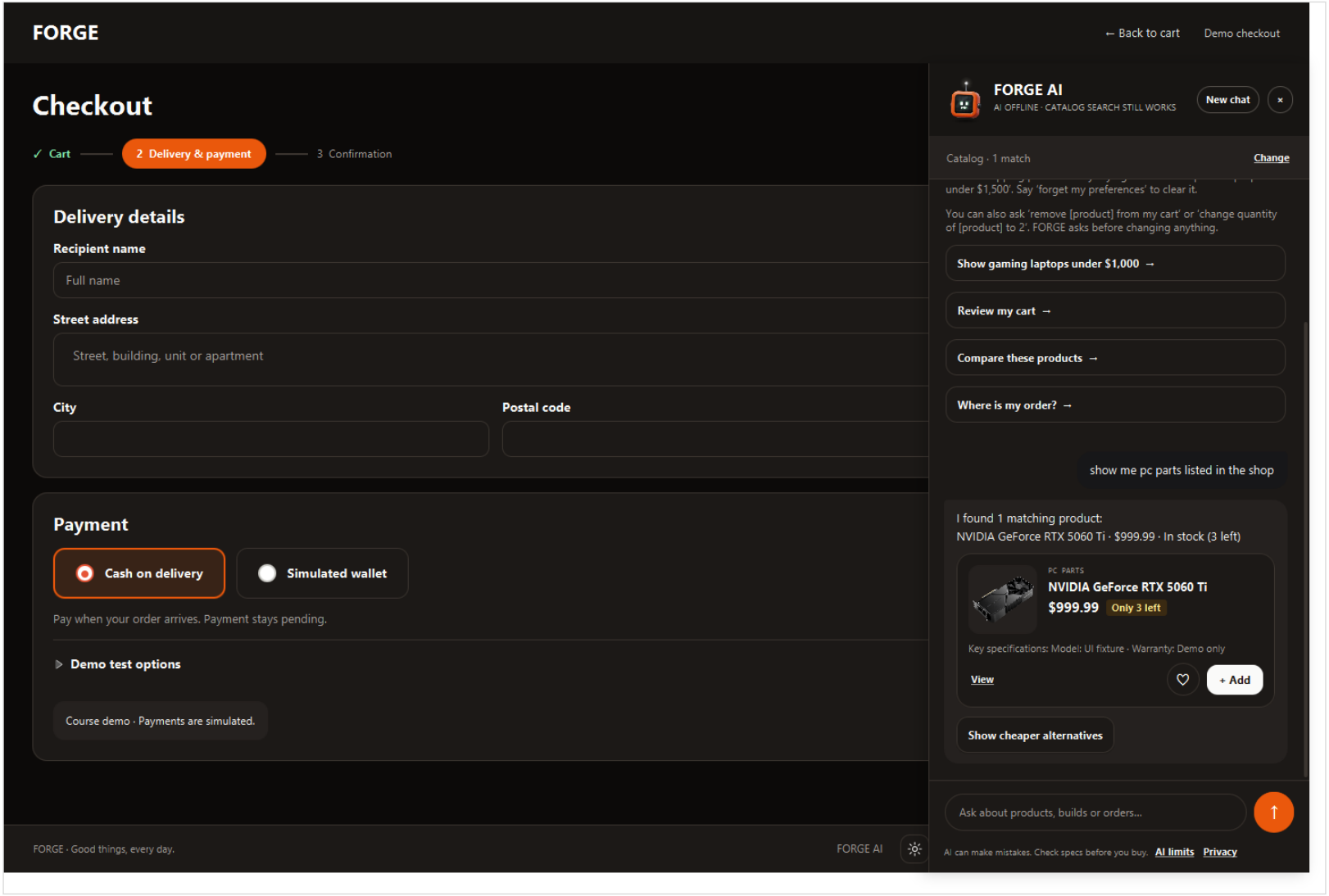
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



Administrators inspect recorded orders and manage simulated fulfillment. Service checks prevent invalid transitions and duplicate stock restoration. Verification: AdminDashboardUiSmoke.

Figure 11 FORGE AI customer assistant

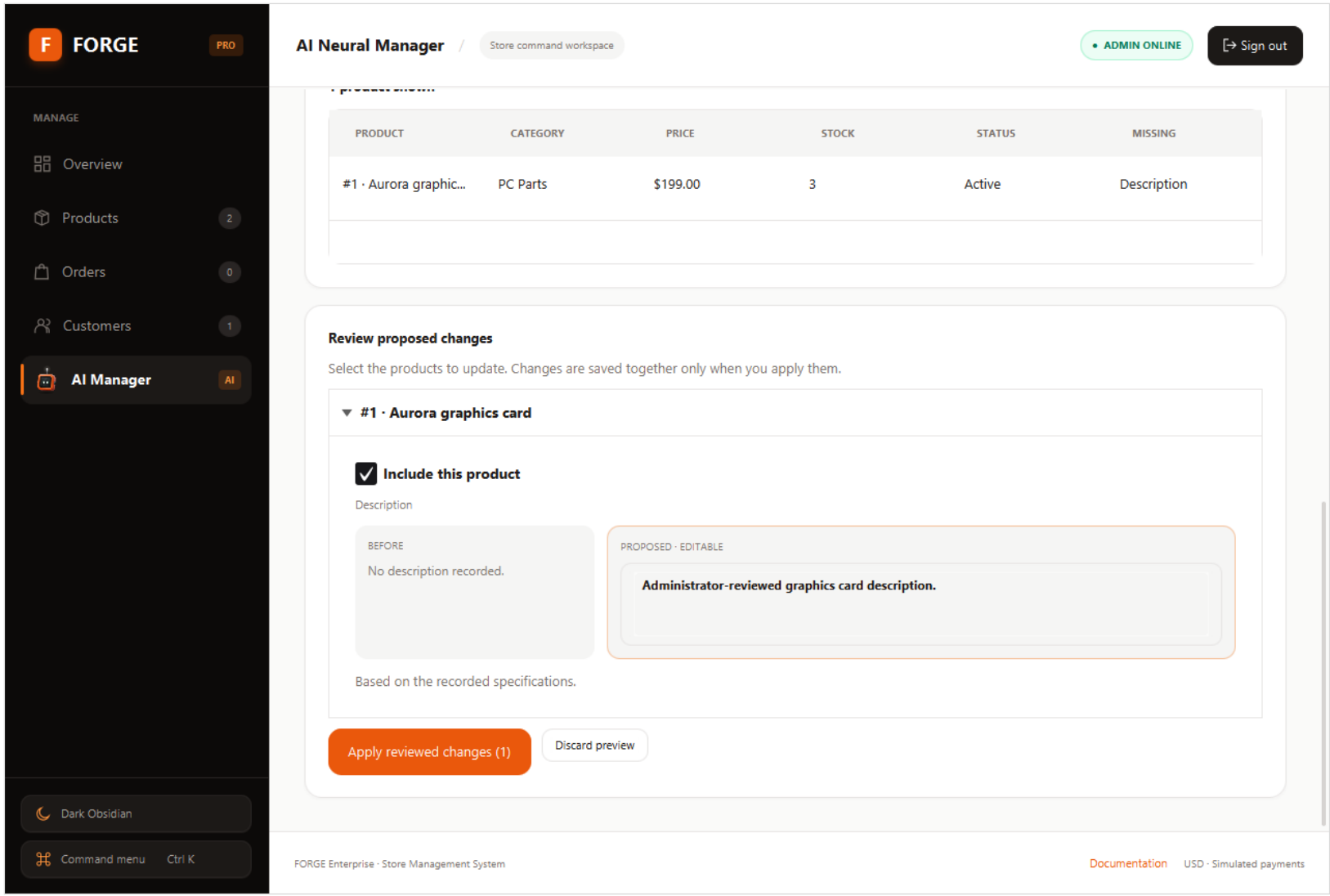
Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



A local catalog query renders product cards inside the conversation. The current local records determine the cards; product specifications are optional. Verification: StorefrontUiSmoke.

Figure 12 AI Manager reviewed content preview

Section 8 | Actual JavaFX scene | Isolated verification data | 24 October 2026



This deterministic provider fixture demonstrates editable generated drafts before Apply reviewed changes. The test verifies that previewing does not write to SQLite. The capture variant expands the editable draft and scrolls the review into view; it uses the same assertions. Verification: AiManagerUiSmoke with AiManagerDocumentationCapture.

9. Testing

Verified results

On 24 October 2026, build.ps1 Test completed with BUILD SUCCESS using JDK 26.0.1, Java 21 source release and JavaFX 21.0.6 on Windows. All 175 JUnit tests passed: zero failures, zero errors and zero skipped tests. Eleven current test suites were counted; obsolete report files from removed test classes were excluded from the summary.

Seven additional JavaFX suites passed 121 assertions against actual controls: storefront 19, AI Manager 19, AI quota feedback 12, commerce 15, customer navigation 22, login glass 18 and admin dashboard 16. These are UI assertions, not 121 additional JUnit test cases. Tests used fresh databases and isolated home directories; they did not read the production store database or use a live AI key.

JUnit suite	Tests	Failures	Errors	Skipped
AccountAndRepeatTest	9	0	0	0
CommerceWorkflowTest	28	0	0	0
service.AiManagerServiceTest	25	0	0	0
service.AiQuotaTest	13	0	0	0
service.AiReliabilityTest	19	0	0	0
service.CatalogAssistantTest	24	0	0	0
service.GeminiAssistantTest	7	0	0	0
service.RevenueAnalyticsTest	9	0	0	0
ShopTest	35	0	0	0
TechCategoriesTest	3	0	0	0
ui.AssistantPanelTest	3	0	0	0

Important success and failure cases

- Authentication resolves the correct role; wrong credentials and unauthorized administrative actions are rejected.
- Invalid names, prices, stock, quantities, photos, reviews and supported specification values are rejected without corrupting saved records.
- Approved wallet and cash-on-delivery orders commit their intended payment state. Declined payments, insufficient stock and injected database failures roll back the entire checkout.
- Receipt snapshots survive product edits and deletion. Cancellation restores stock only once; invalid transitions and stale edits fail safely.

- Saved carts remain account-scoped, restore with current price and stock checks and merge guest choices. Login opens the storefront. Favoriting preserves scroll and focus.
- Product cards render from catalog matches even when optional specifications are absent. PC parts and PC sets are distinct categories; supported corrections and exclusions are covered.
- AI Manager validates tool arguments, preserves existing content, requires reviewed changes, rejects stale batches and prevents repeat application. Provider failures cannot fabricate draft edits.
- Quota parsing uses structured provider metadata. Tests distinguish minute, daily, token and unknown restrictions, retry hints, zero allowances and redaction of private metadata.
- Revenue aggregation uses exact cents, zero-filled dates and the selected time zone. Unpaid orders are excluded from the paid metric.

Evidence and reproduction

Raw test output, UI assertion logs, screenshot provenance and verification.json are in docs/evidence/2026-10-24. The current JUnit XML reports are generated in target/surefire-reports. Source hashes in the UML inventory identify the Java version documented here; later code changes require fresh verification.

From a PowerShell terminal in the project directory, run .\build.ps1 Test. This action does not import the saved AI key. The ordinary Run action and the existing batch launcher start the application. Maven needs a working JDK 21 or newer, its dependencies and an appropriate JavaFX platform runtime. The normal Maven JAR is not a standalone Windows installer.

For native UI verification, first run Maven test-compile and launch a named smoke class with target/classes, target/test-classes and the runtime dependencies on the classpath. Each suite takes an evidence directory. The exact suites, isolation settings and commands are recorded in TESTING.md. Mock provider fixtures verify application behavior; they do not certify Google's service uptime, model quality or the account's remaining quota.

Manual checks before presentation

The group must still rehearse on the presentation computer: launch the batch file; check display scaling and resizing; navigate with the keyboard; open and cancel the native photo chooser; upload representative images; test a live AI request when quota is available; and verify reduced motion and robot animations on the actual graphics hardware. Complete the code walkthrough individually. These checks are pending personal presentation preparation, not recorded automated passes.

The group should be ready to trace CheckoutView to CheckoutService, ProductRepository, PaymentMethod and OrderRepository, and to explain the review boundary between AiManagerService.prepare and apply. Be prepared to modify a small validation rule, update its test and explain why changing only the UI would not be enough.

10. Conclusion

FORGE demonstrates how object-oriented design connects a user interface to consistent business operations. Encapsulation protects product and cart state; inheritance represents roles with different permissions; polymorphism selects payment behavior at runtime; abstraction keeps checkout dependent on a contract. JDBC transactions preserve the relationship between inventory, saved carts and purchase records.

The project also demonstrates the practical limits of assisted software. Product and order facts come from local records, generated content is reviewed before saving, and incomplete specifications are not treated as verified compatibility. Provider errors and quotas are surfaced separately from local store operations. Testing covers both successful workflows and deliberate failures, while the UML and source inventory make the implementation traceable.

The implementation provides a working course demonstration rather than a production commerce service. Future work could include a packaged runtime, a secured remote service, account registration and real provider integrations with their own recovery and audit requirements. Before submission, each group member must review this explanation, confirm that it reflects their own understanding and rehearse the required defense. Instructor approval of the title and presentation performance are confirmed by the group and instructor, not by automated tests.